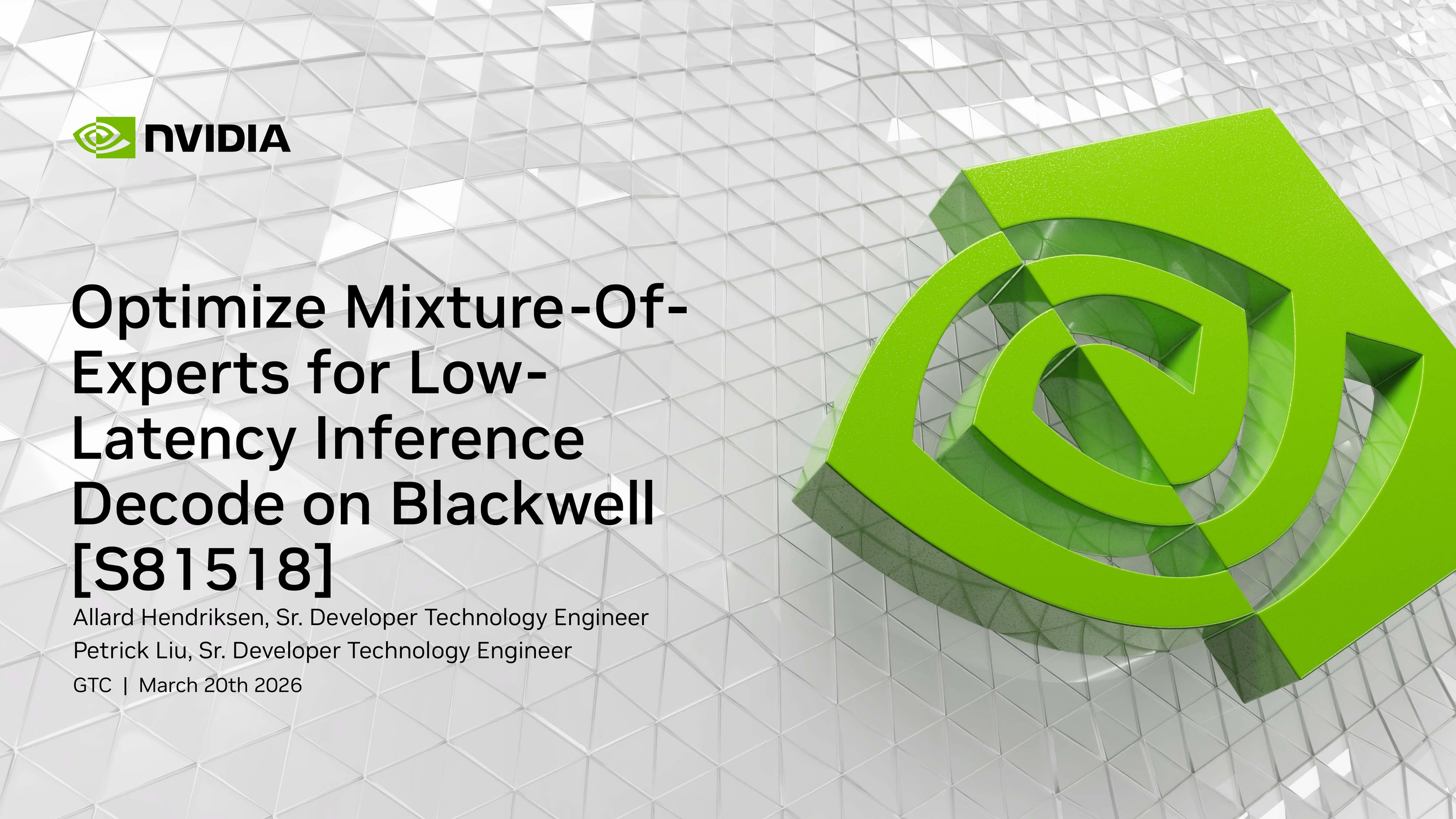


Optimize Mixture-Of-Experts for Low-Latency Inference Decode on Blackwell [S81518]

Allard Hendriksen, Sr. Developer Technology Engineer
Petrick Liu, Sr. Developer Technology Engineer

GTC | March 20th 2026

Agenda

Introduction to LLM Inference and MoE

Blackwell Tensor Core

System-level Optimizations

Warp-specialized kernel design

Latency-optimized warp-specialization

Kernel-level optimizations

Introduction to LLM Inference and Mixture of Experts

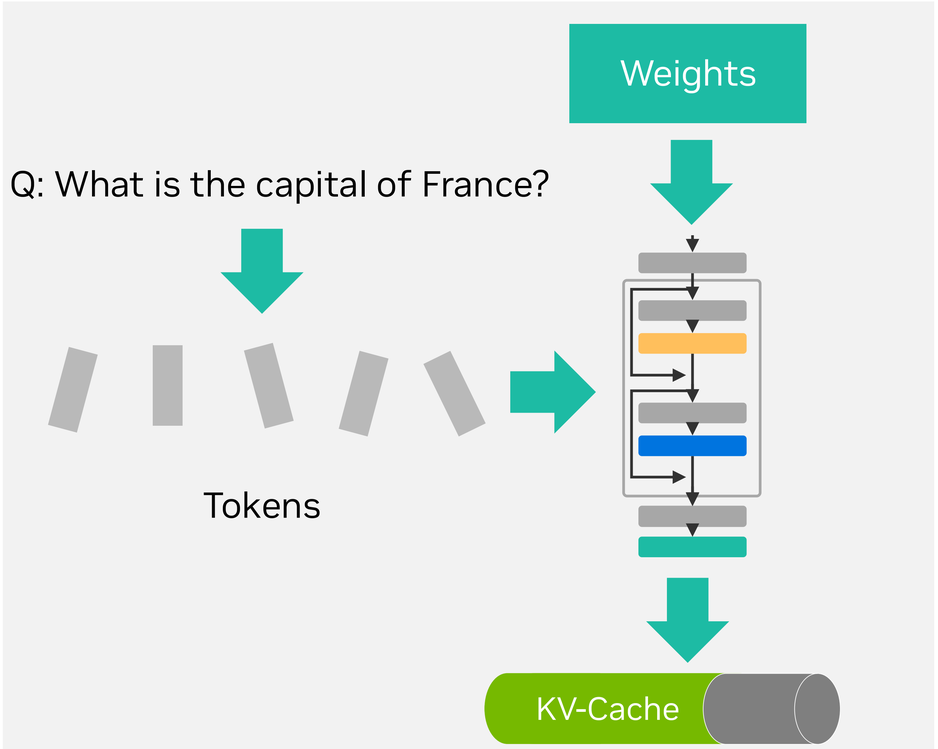
LLM Inference on Blackwell

Q: What is the capital of France?

A: The capital of France is Paris.

Context phase and generation phase

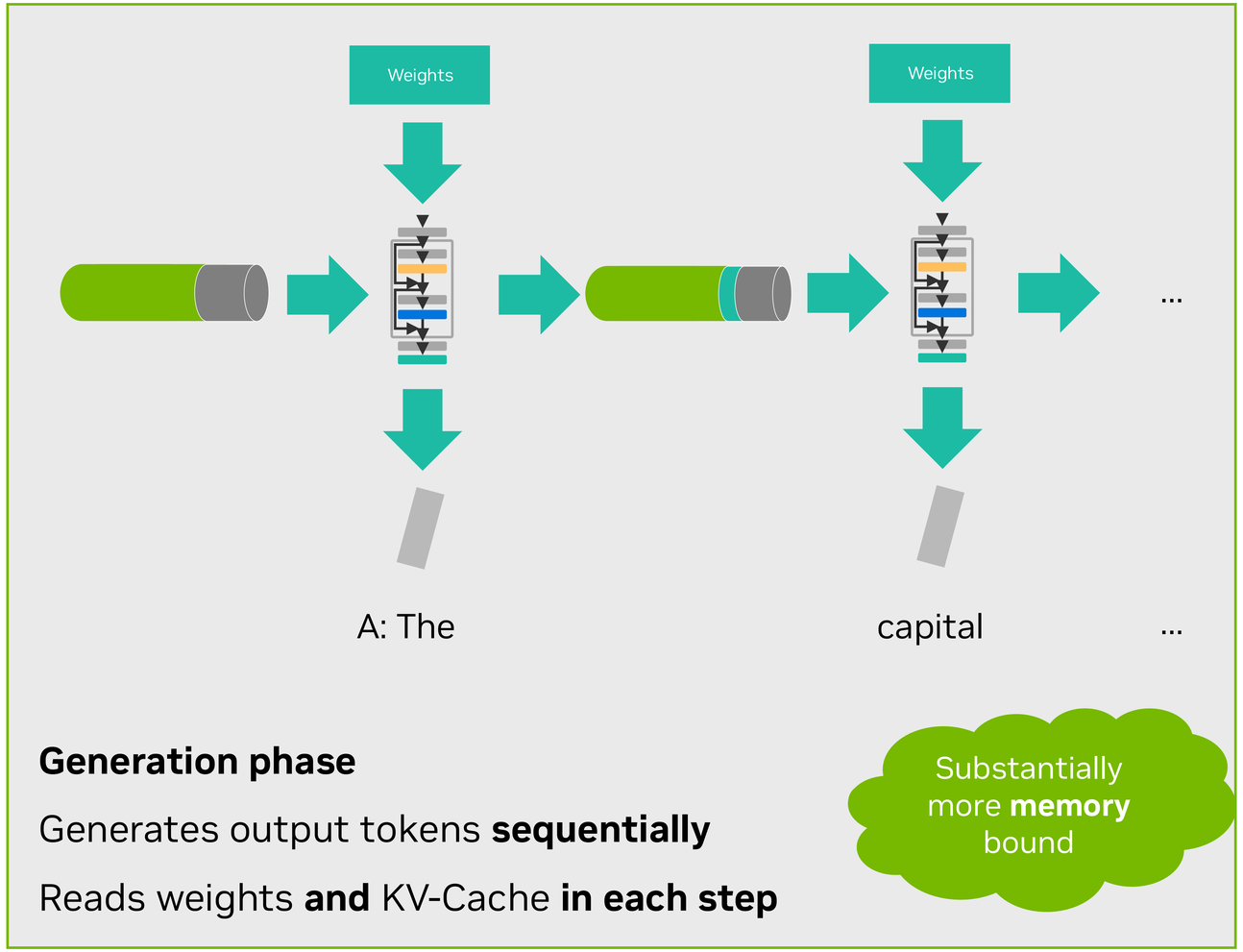
Listening and answering



Context phase

Processes all input tokens in **parallel**

Reads weights **once**



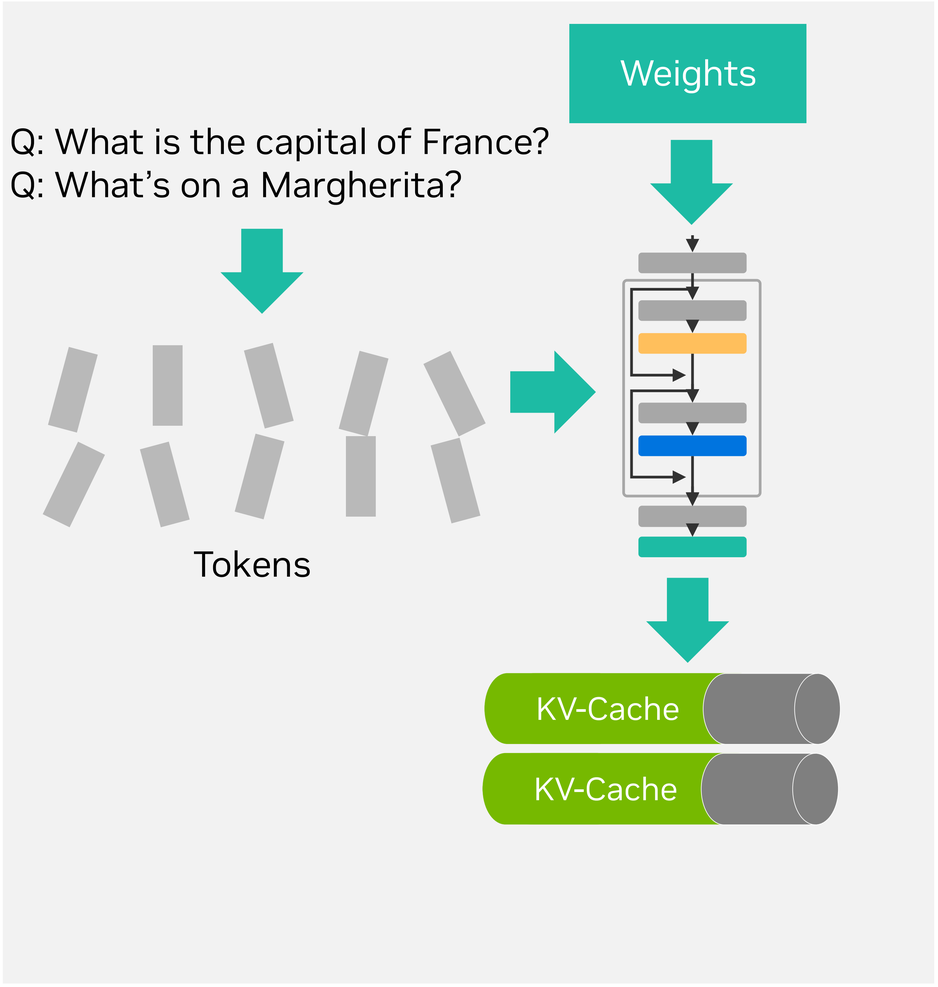
Generation phase

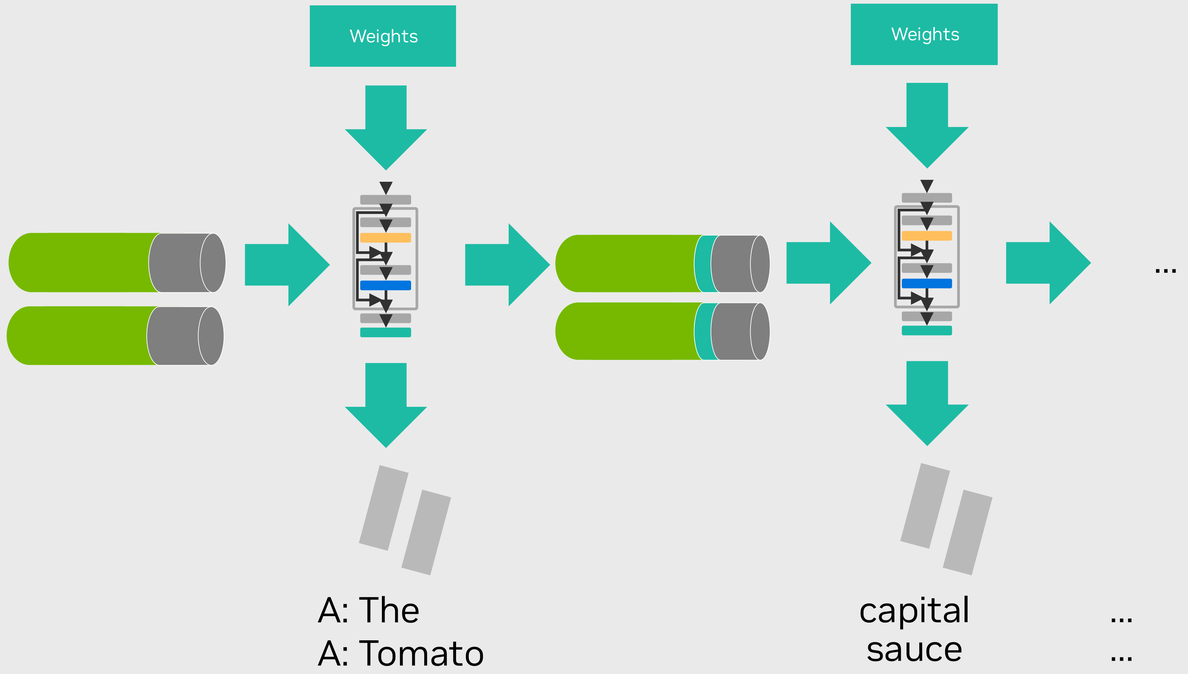
Generates output tokens **sequentially**

Reads weights **and** KV-Cache **in each step**

Batching

Process multiple queries in parallel

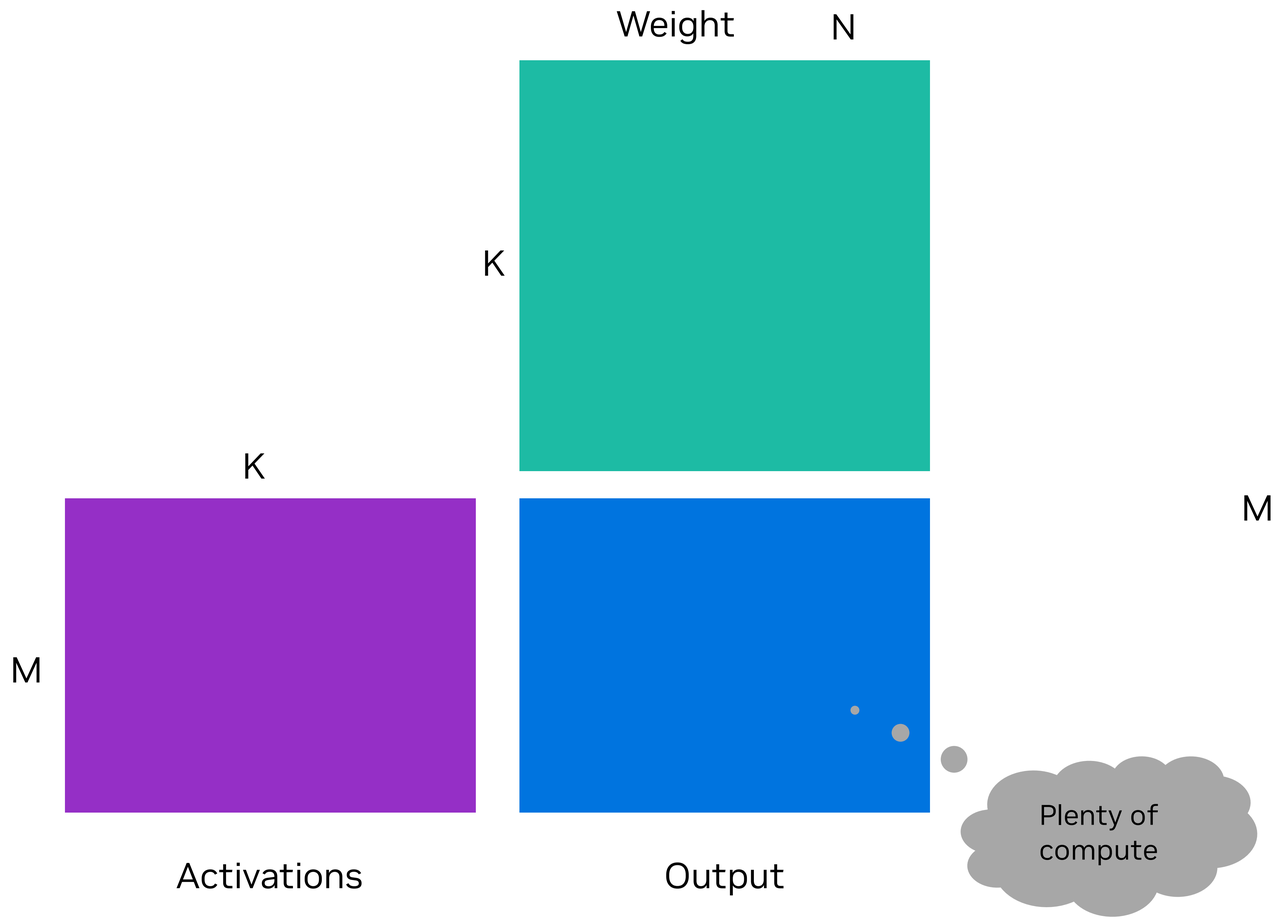




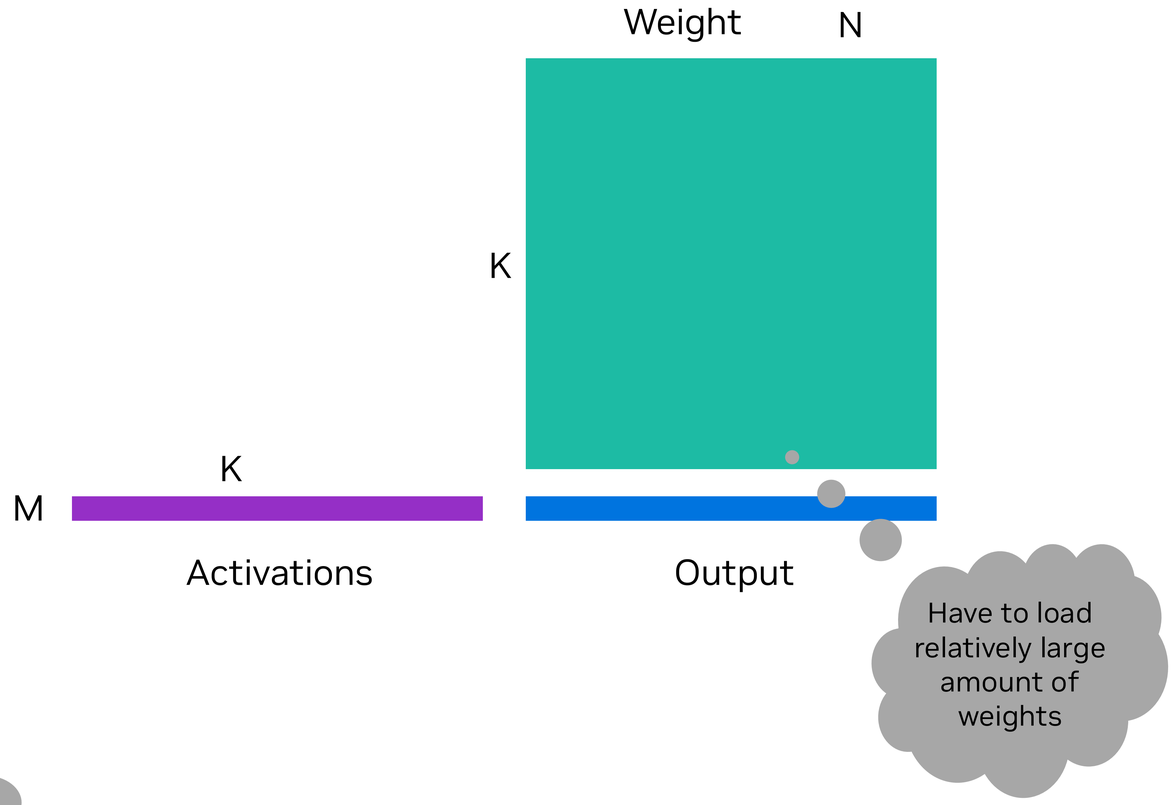
Batching

- Increases # of generated tokens per generation step
- Decreases bandwidth pressure of weight loads

Impact of batch size on GEMM

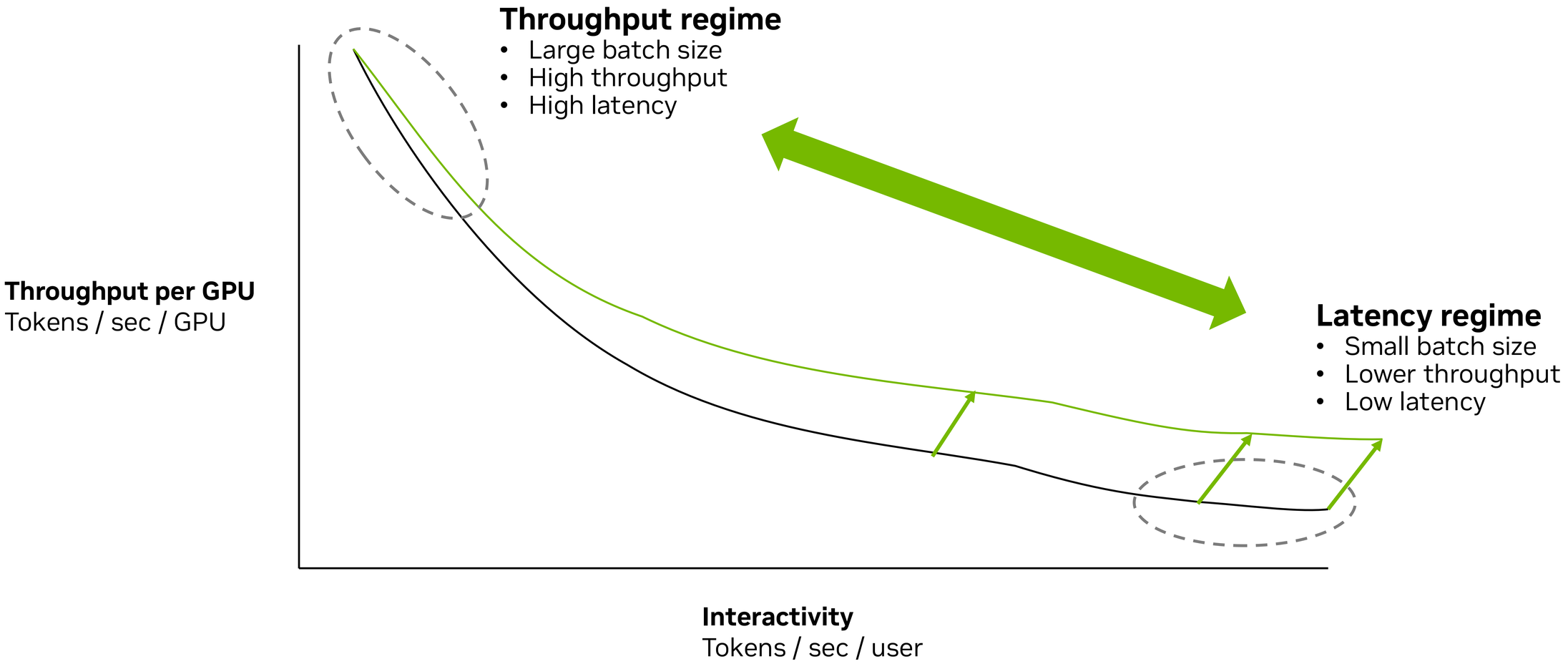


Large batch size



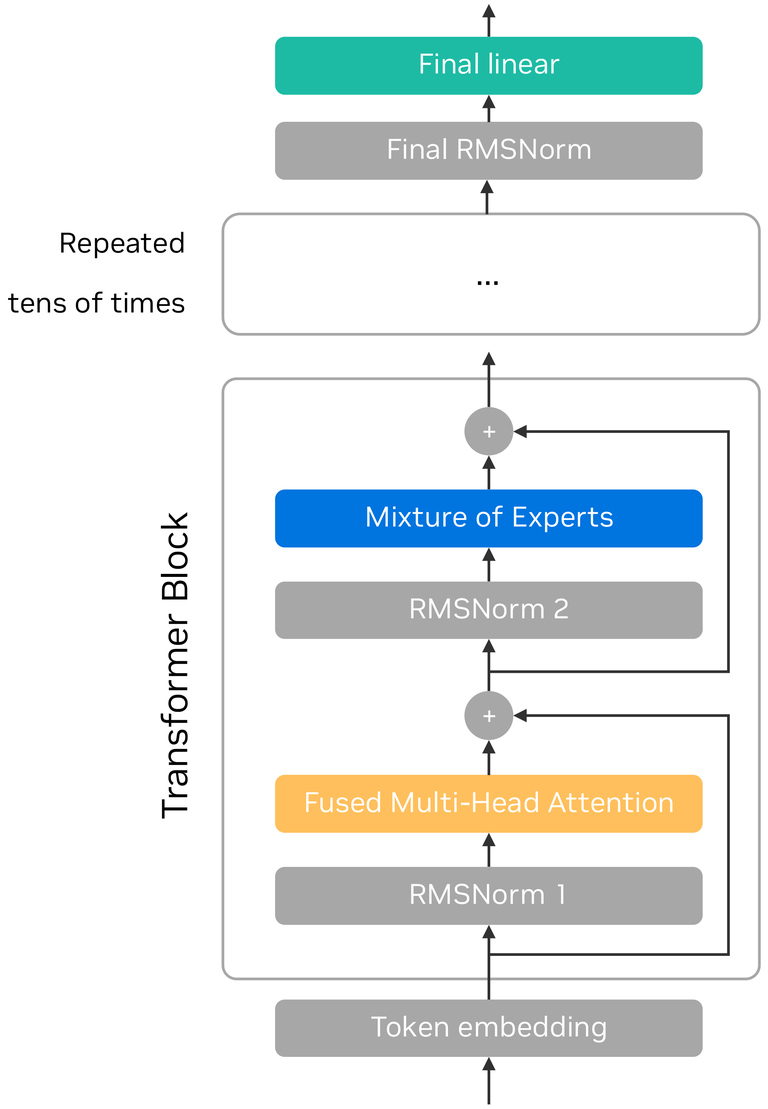
Small batch size

Throughput versus latency



Anatomy of an LLM

- LLMs consist of tens of transformer blocks
- Transformer blocks contain Attention and Mixture-of-Experts
- Mixture-of-Experts takes **50-80%** of time on small output sequences (<1K)



Courtesy of Louis Sugy

Anatomy of Mixture-of-Experts (MoE)

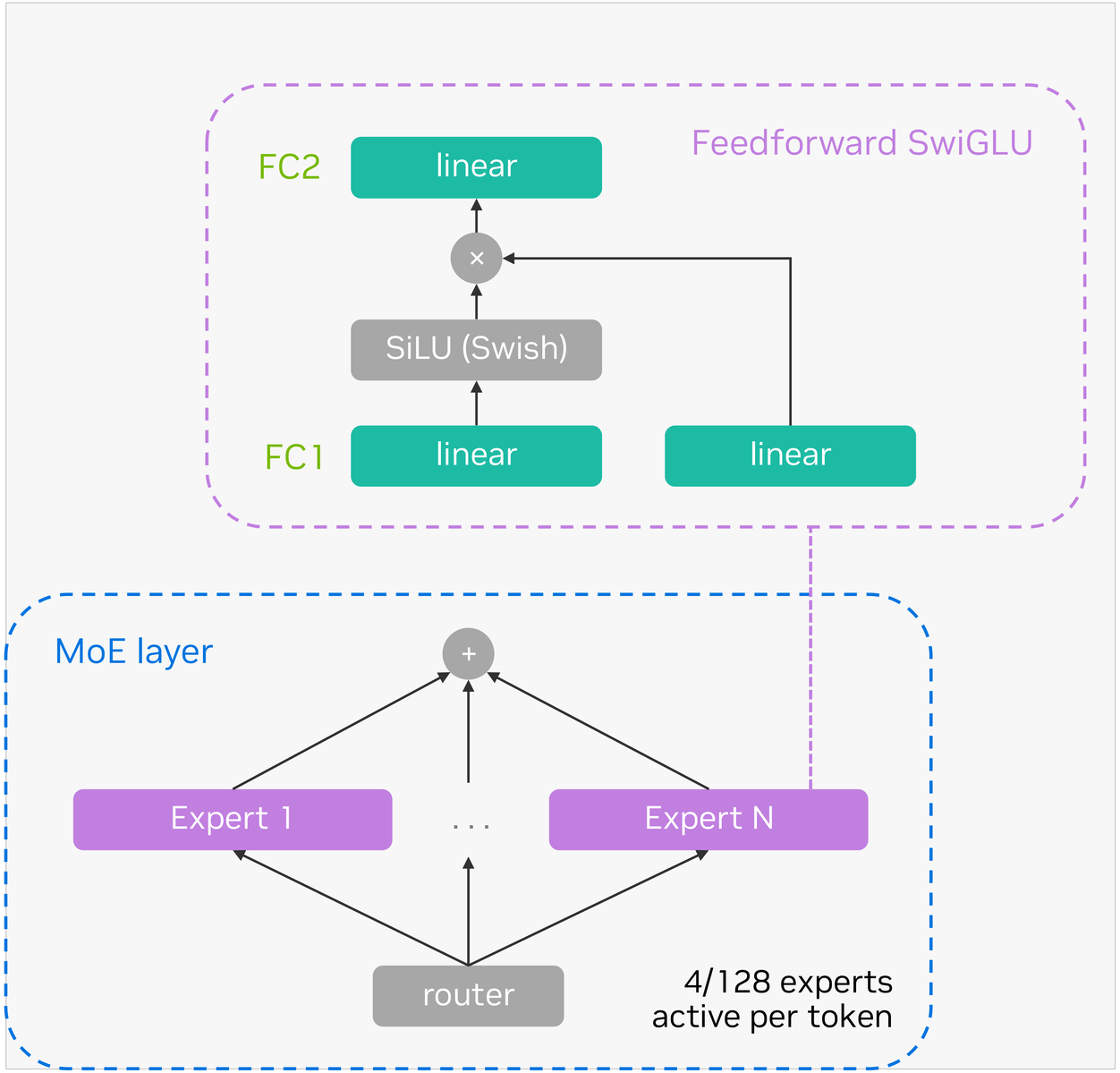
An MoE layer consists of hundreds of experts

Each token gets “routed” to a small subset of experts (4)

Each expert consists of

- A gated linear layer – 2 GEMMs + activation (FC1)
- A linear layer (FC2)

Output is weighted sum of experts



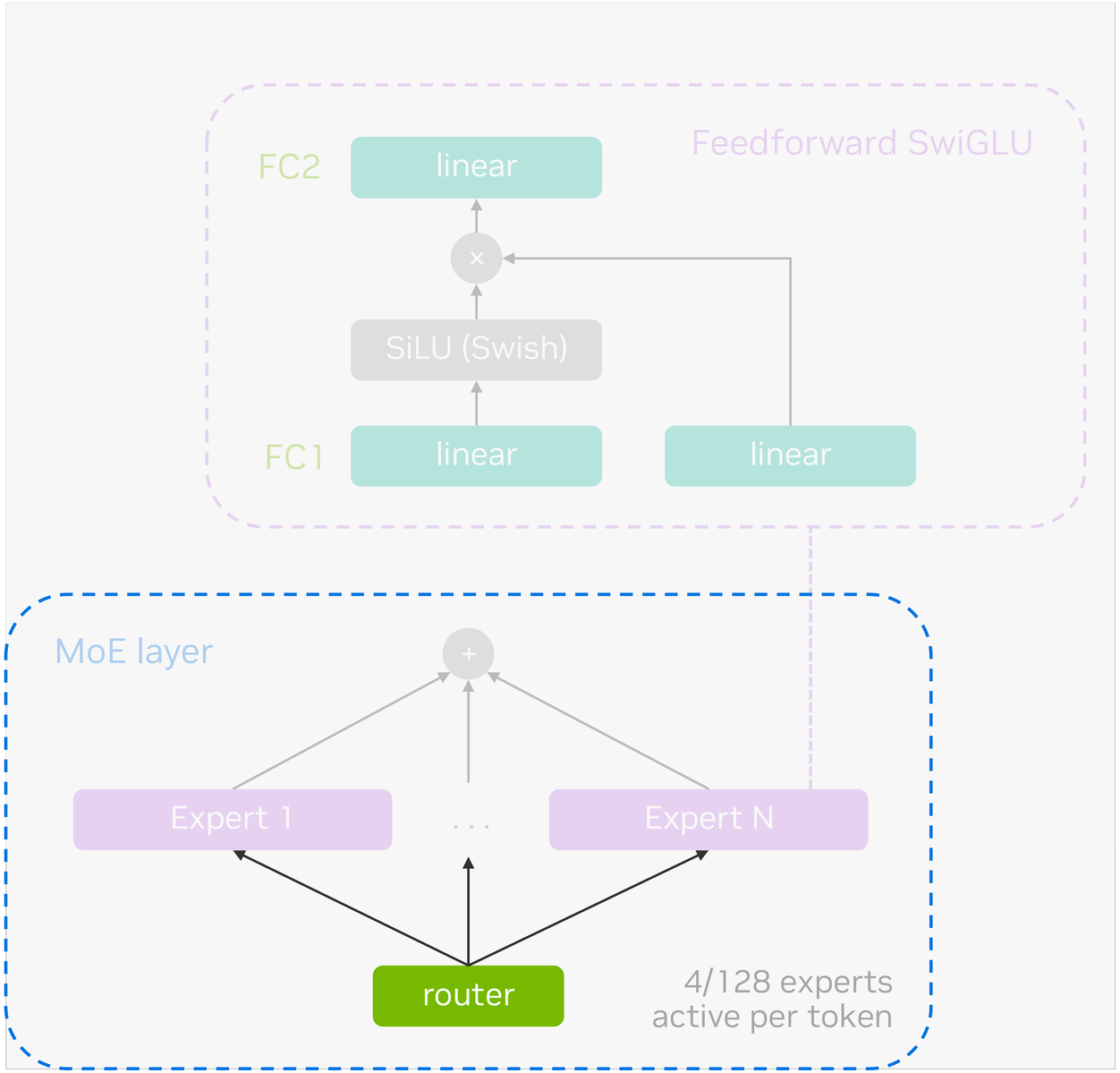
Courtesy of Louis Sugy

Routing

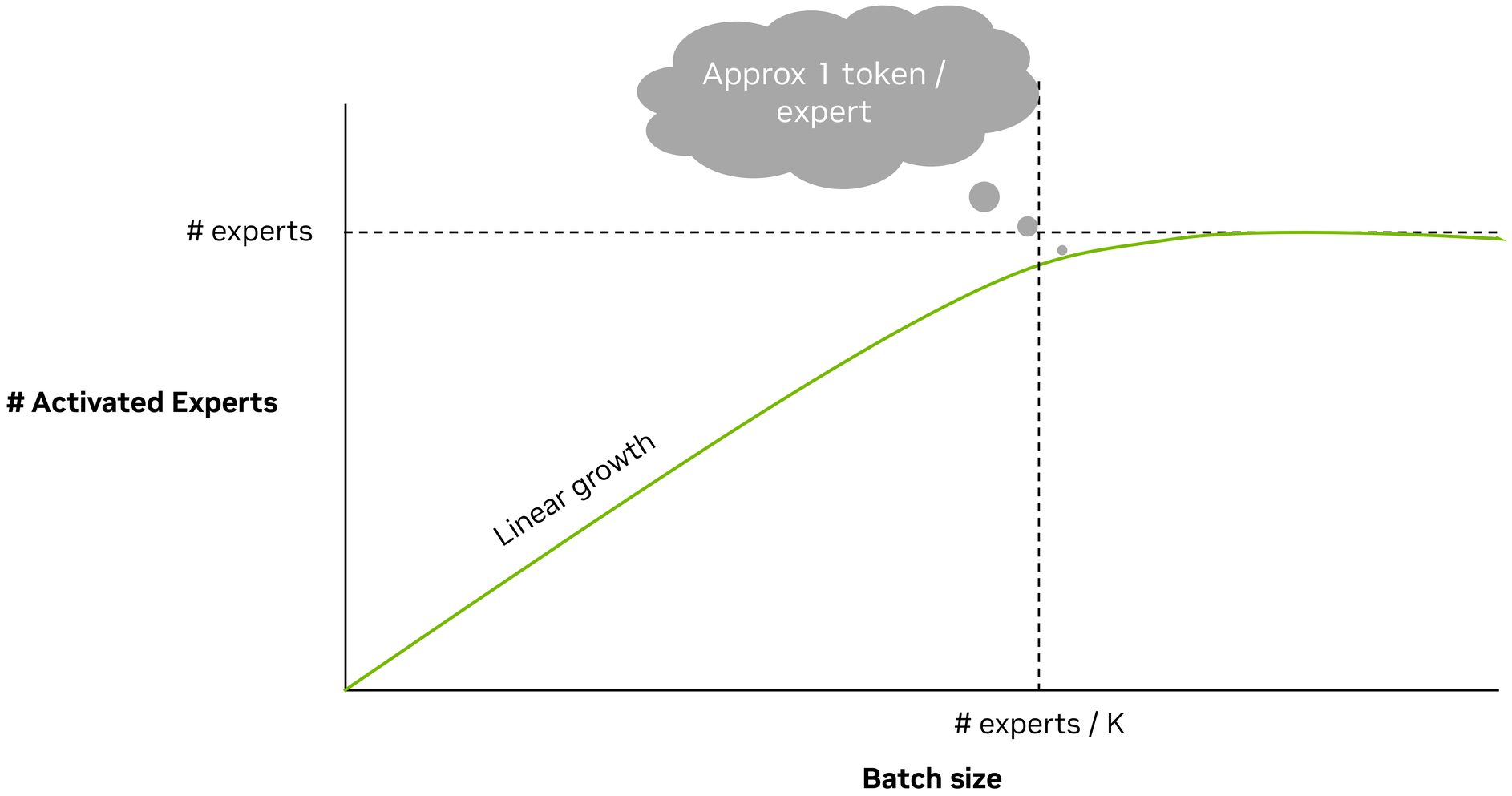
Each (token, expert) combination gets a score

Score is computed by a small GEMM

Per token: top-K scores determine K activated experts



Number of activated experts



Many kernels...

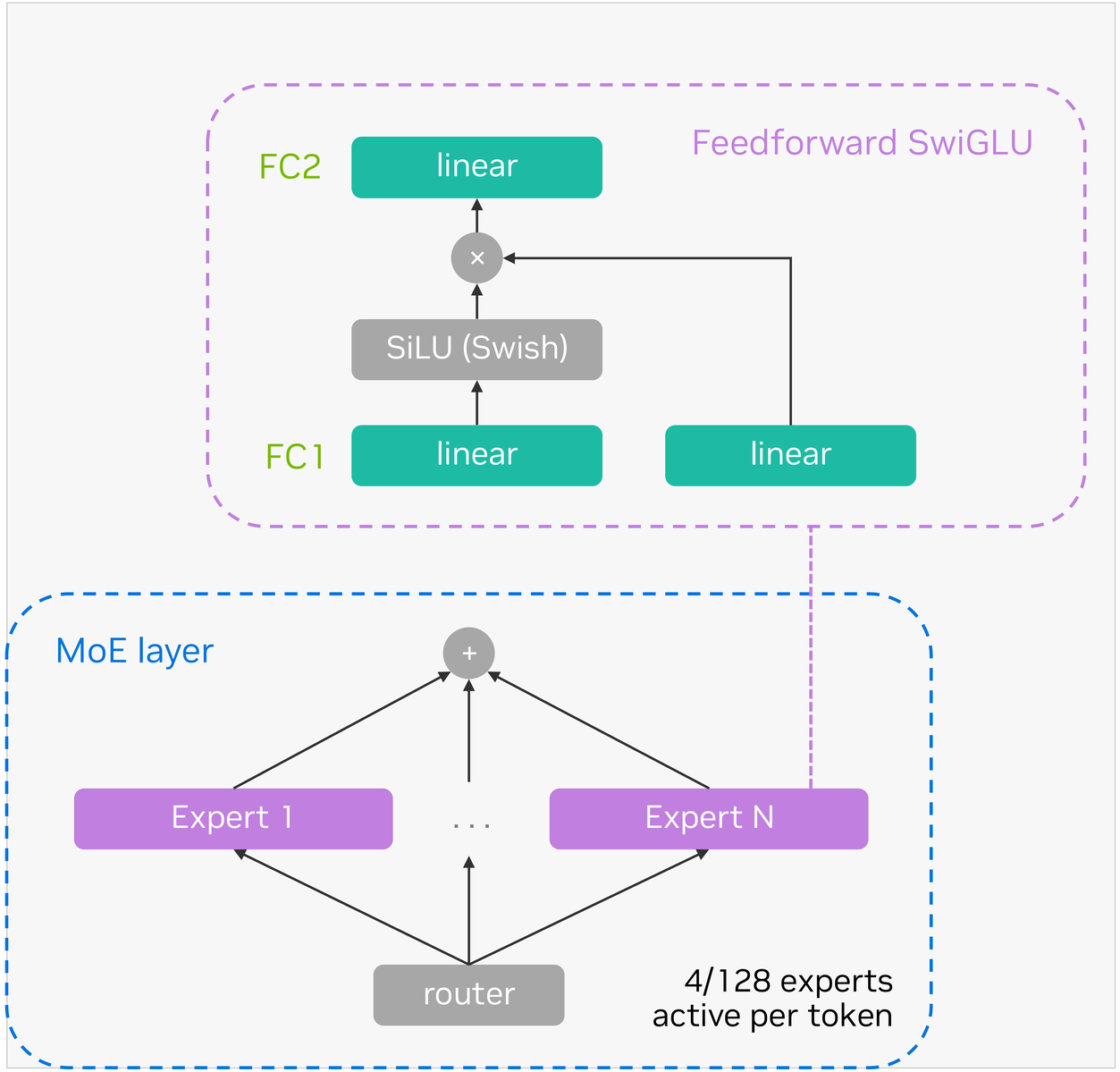
100 experts * 3 GEMMs / Expert ➡ 300 GEMMs / MoE

1 us / kernel ➡ 300 us per MoE

100 layers ➡ 30ms / per LLM step

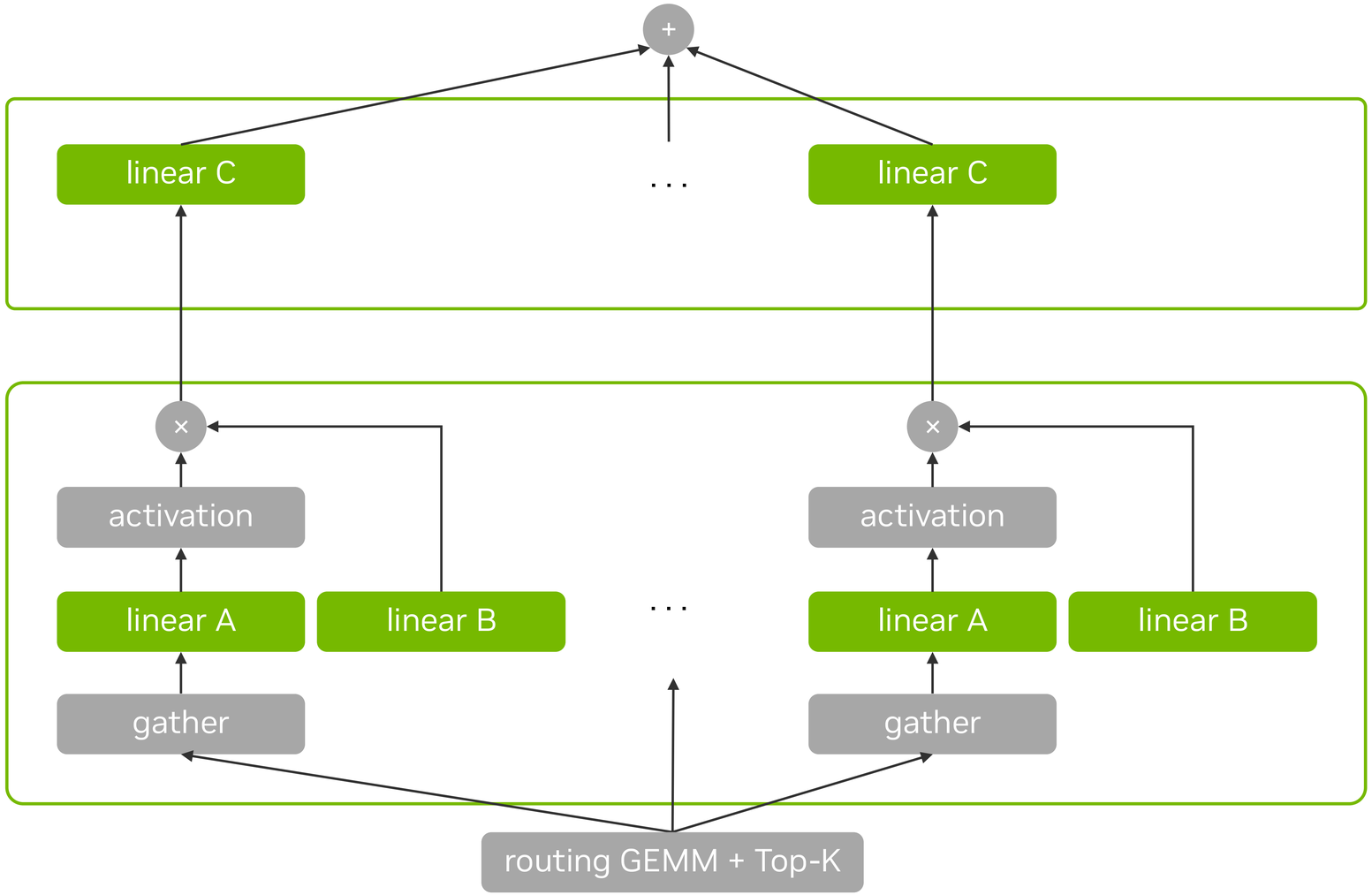
❌ Maximum of **30 tokens / second**

➡ We need to **fuse** kernels



Fusing Mixture of Experts

2 kernels



Batched GEMM FC2

- Grouped by expert
- All linear layers C are computed in the same kernel

Batched GEMM FC1 with fused gather and gated activation

- Grouped by expert
- Gathers tokens routed to this expert
- Linear layers A and B are computed in the same kernel
- Activation is computed in the epilogue

Low-latency Mixture-of-Experts

Challenges

Memory bound

- Read weights every generation step
- Small batch size

Non-square GEMM

- Small batch size
- Split per expert
  - Even smaller M dimension

Short kernels

- Kernel runtimes become shorter
- Amdahl's Law: sequential latencies start to dominate
  - Launch latency
  - Prologue latency

Non-contiguous data

- Want to avoid redundant data movement
- Incorporate gather/scatter into GEMM kernel

Blackwell Tensor Cores

Blackwell Tensor Core

tcgen05.mma family with Tensor Memory

Data type: TF32, BF16, FP16, {MX}FP8, {MX}FP6, {MX}FP4, NVFP4

Accumulators are in TMEM.

Operand A in TMEM or SMEM.

Operand B in SMEM

Scaling factors are in TMEM

INST_N = [8, 256],

INST_K = 32B (sm_100f), 48B(sm_103a, {MX,NV}FP4)

eg: FP16/BF16 INST_K = 16;

FP8. INST_K = 32

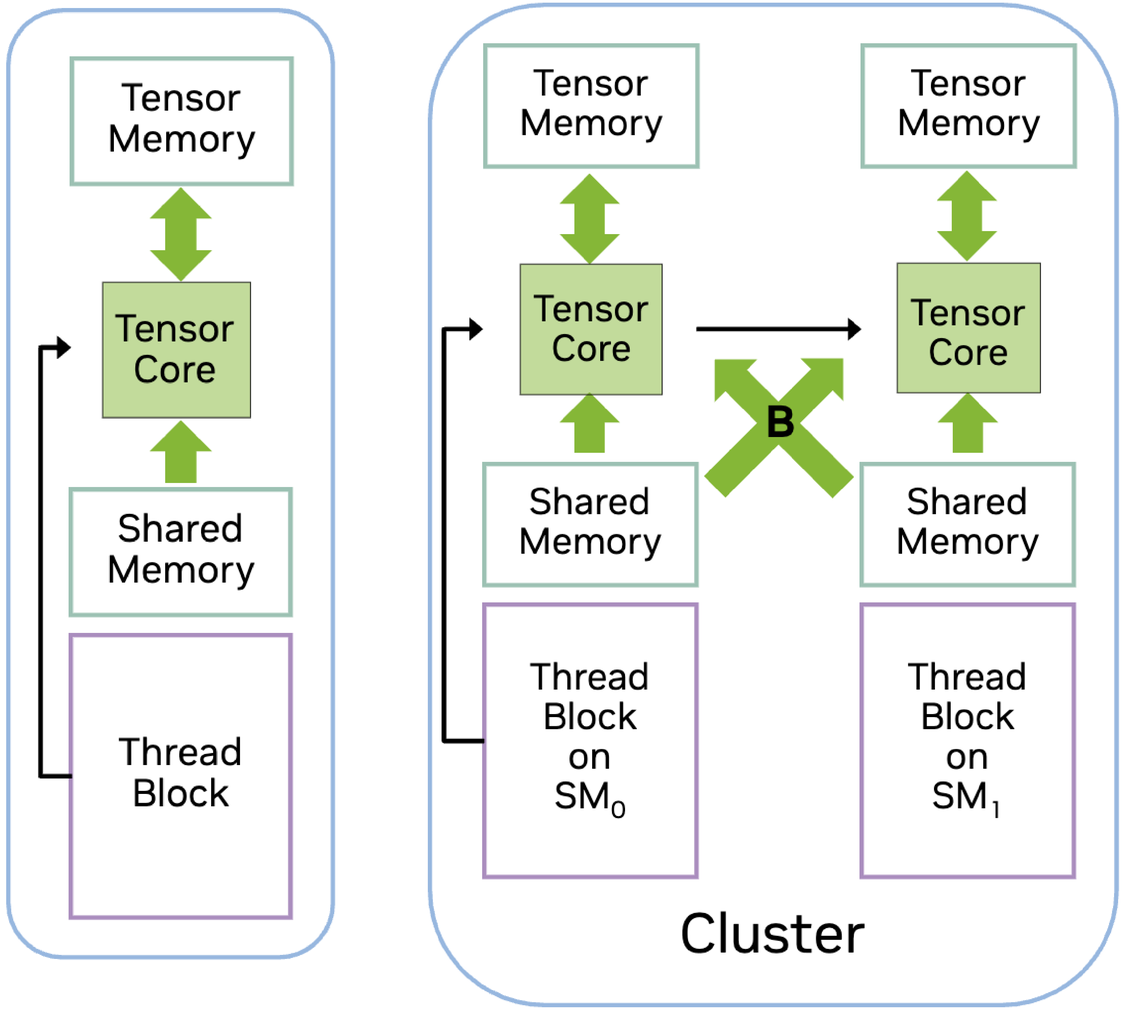
1 thread in CTA/CTA-pair submits the tensor core async execution.

1CTA mode:

- INST_M = 64, 128

2CTA mode:

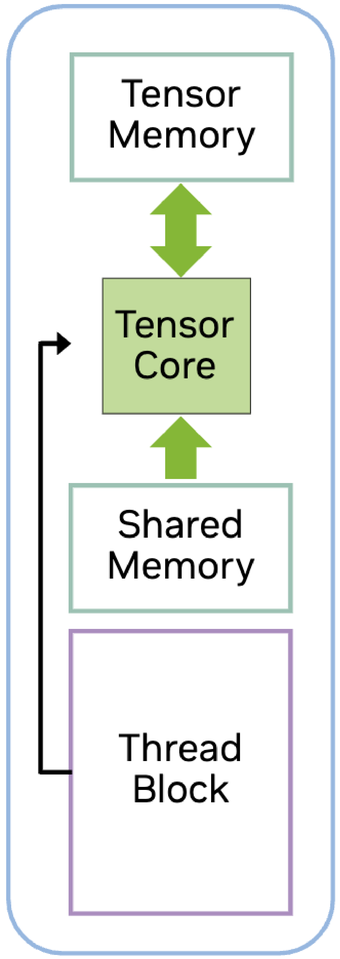
- INST_M = 128, 256

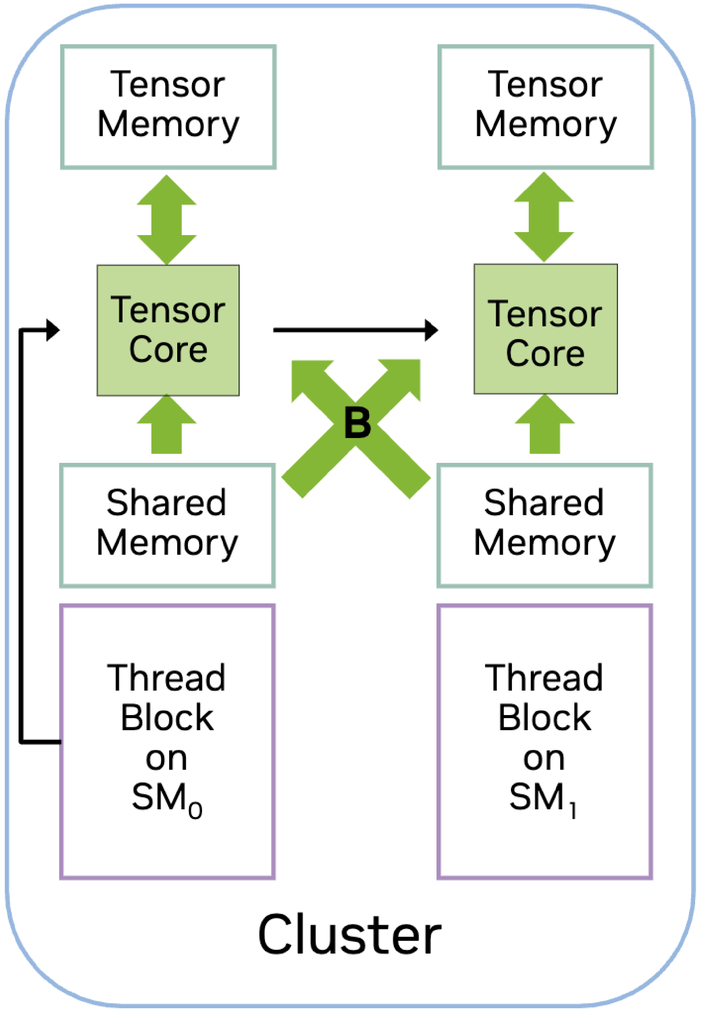


Blackwell Tensor Core

tcgen05.mma family with Tensor Memory

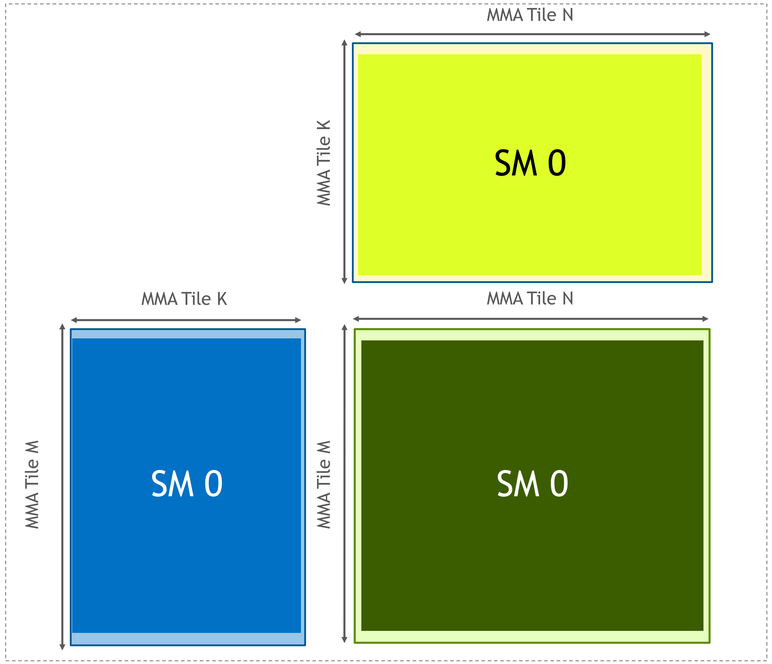
```
// 1. Floating-point type without block scaling:

tcgen05.mma.cta_group.kind   [d-tmem],  a-desc,  b-desc, idesc,
                             { disable-output-lane }, enable-input-d {, scale-input-d};

tcgen05.mma.cta_group.kind   [d-tmem], [a-tmem], b-desc, idesc,
                             { disable-output-lane }, enable-input-d {, scale-input-d};

.kind      = { .kind::f16, .kind::tf32, .kind::f8f6f4 }
.cta_group = { .cta_group::1, .cta_group::2 }

-------------------------------------------------------------------------------

// 2. Floating-point type with block scaling:

tcgen05.mma.cta_group.kind.block_scale{.scale_vectorsize}
                                     [d-tmem],  a-desc,  b-desc, idesc,
                                     [scale-A-tmem], [scale-B-tmem], enable-input-d;

tcgen05.mma.cta_group.kind.block_scale{.scale_vectorsize}
                                     [d-tmem], [a-tmem], b-desc, idesc,
                                     [scale-A-tmem], [scale-B-tmem], enable-input-d;

.kind = { .kind::mxf8f6f4, .kind::mxf4, .kind::mxf4nvf4 }
.cta_group      = { .cta_group::1,   .cta_group::2 }
.scale_vectorsize = { .scale_vec::1X, .scale_vec::2X, .scale_vec::4X, .block16, .block32 }

-------------------------------------------------------------------------------
```





Blackwell Tensor Core

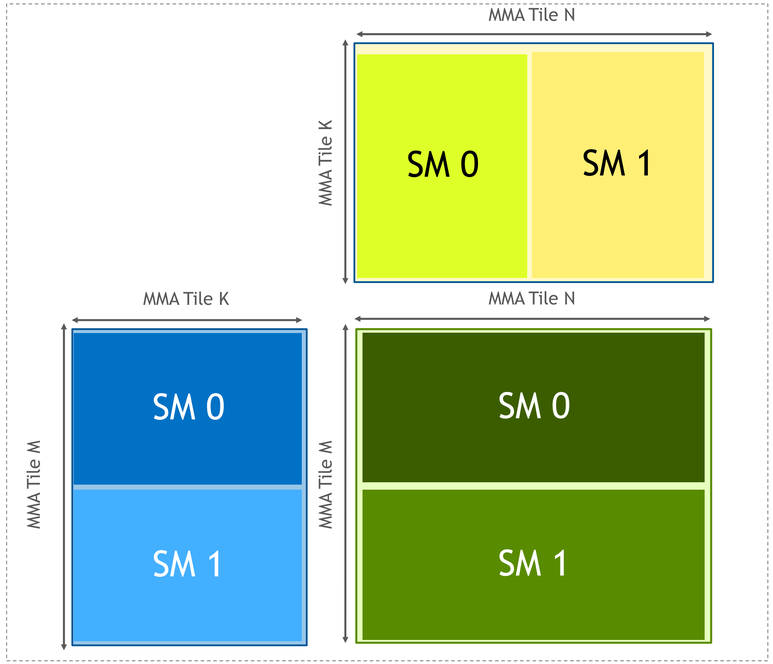
Data distribution in 1CTA & 2CTA mode



1 CTA mode:

CTA holds the whole matrixes:

A(Smem or Tmem), B(Smem) and C(Tmem)



2CTA mode:

Each CTA holds the partial matrixes:

Half A(Smem or Tmem). Each holds M/2 x K

Half B(Smem). Each holds N/2 x K, will be exchanged between SMs

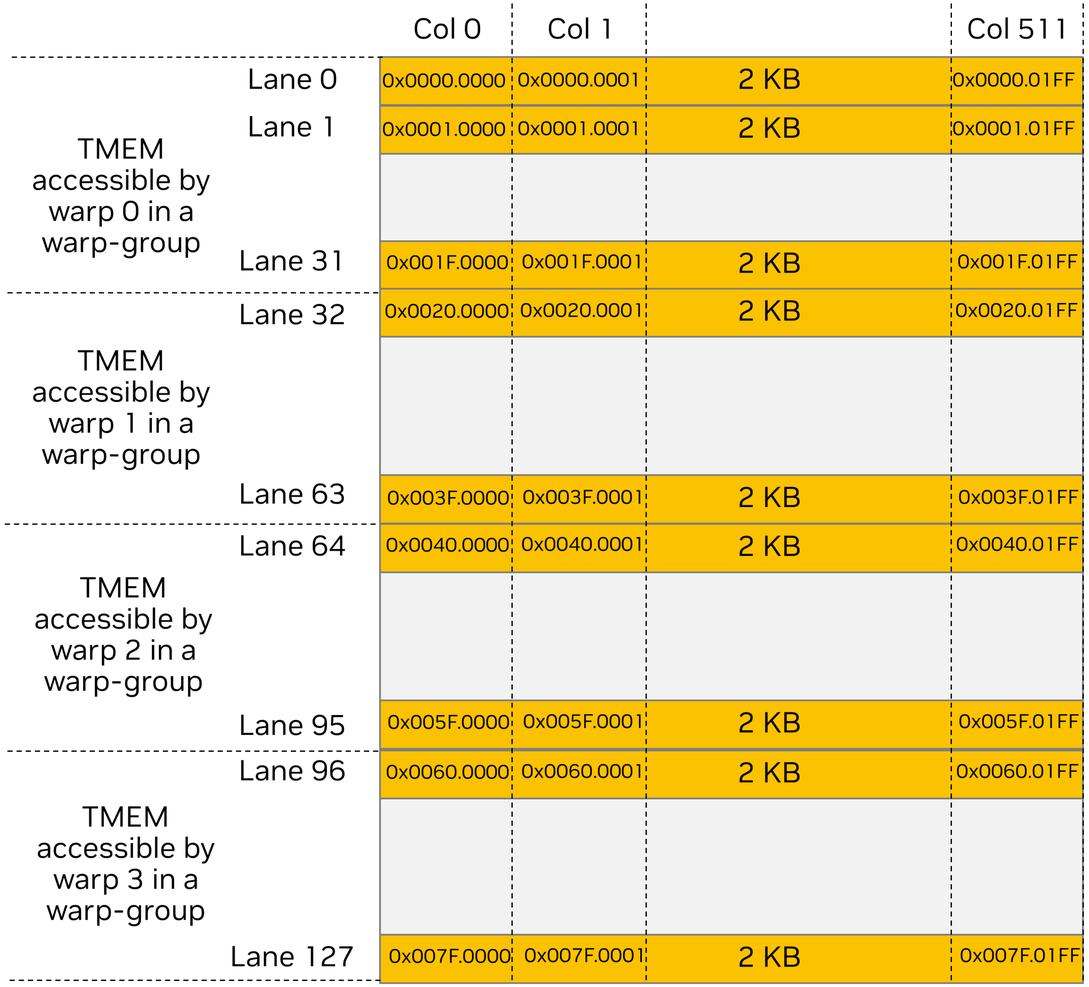
Half C(Tmem). Each holds M/2 x N

Blackwell TMEM and PTX

- TMEM is word-addressed memory.

bits	31… 23	22 …16	15 … 9	8 … 0
encoding	require = 0	TMEM lane	require = 0	TMEM column

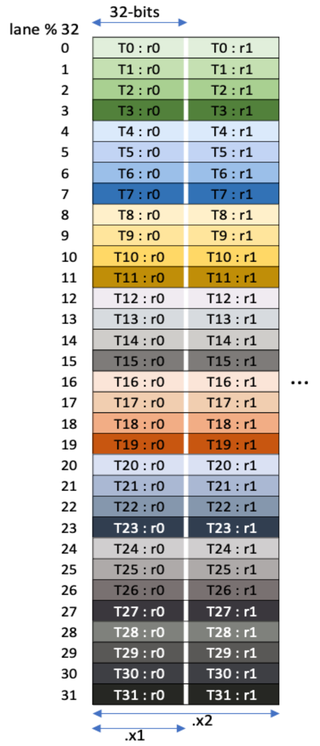
- Addresses consist of 128 "lanes" and 512 "columns".
- Each warp within a warp-group can access only 32 lanes.
- TMEM is explicitly allocated.
- CTAs allocate using tcgen05.alloc PTX
  - Allocation granularity is 128 lanes x 32 columns = 16 KB
  - On allocation, receive a warp-local TMEM address.
- CTAs deallocate using tcgen05.dealloc PTX



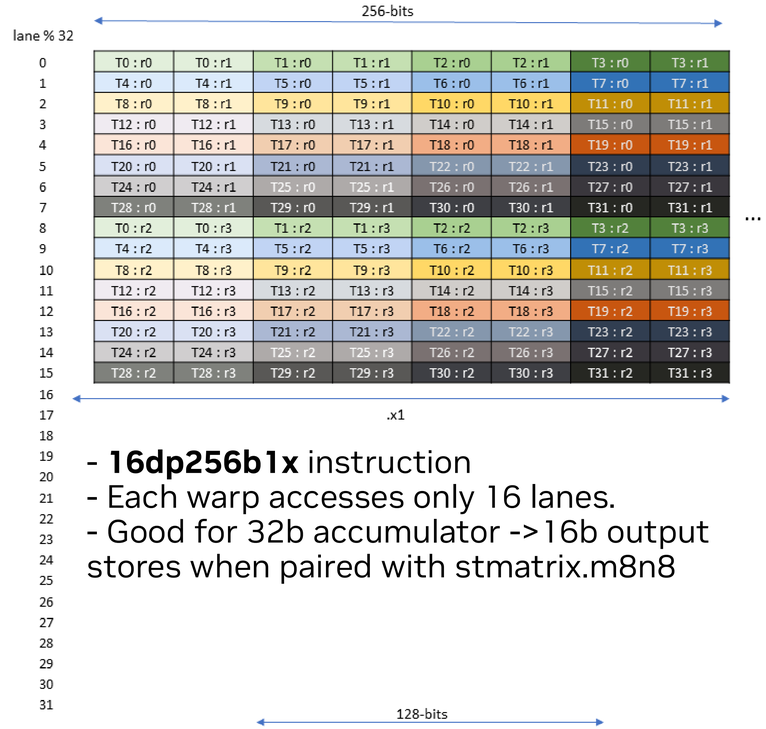
Blackwell TMEM Tensors and Ops

The restrictions of accessing Tmem

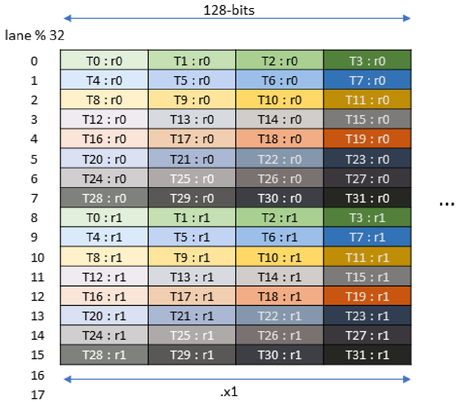
- TMEM addresses can NOT be dereferenced!
- Issue warp-group wide instructions to move data collectively.
  - TMEM ←→ RMEM using tcgen05.load and tcgen05.store PTX.
- 1 thread leads the CTA/CTA-Pair wide data movements.
  - TMEM ← SMEM using tcgen05.cp PTX.
- Access patterns of load/store are fixed by pre-defined layouts.



- **32dp32b2x** instruction
- Each warp access all 32 lanes
- [N]x allows to select num cols
- Good for row major direct stores



- **16dp256b1x** instruction
- Each warp accesses only 16 lanes.
- Good for 32b accumulator ->16b output stores when paired with stmatrix.m8n8



- **16dp128b1x** instruction
- Each warp accesses only 16 lanes.

Leader thread of the warp

All threads in the warp

Blackwell pipelined TC instruction

tcgen05.cp and tcgen05.mma are implicitly pipelined from same thread & same execution scope(1CTA or 2CTA)

A in SMEM, B in SMEM

mbarrier.try_wait(++mbar_smem_full)

tcgen05.mma tmem_accum, ++A_smem, ++B_smem

tcgen05.mma tmem_accum, ++A_smem, ++B_smem

tcgen05.mma tmem_accum, ++A_smem, ++B_smem

tcgen05.mma tmem_accum, ++A_smem, ++B_smem

tcgen05.commit ++mbar_smem_empty

Loop over
all the K iterations

tcgen05.commit mbar_tmem_full

A in TMEM, B in SMEM

mbarrier.try_wait(++mbar_smem_full)

tcgen05.cp tmem_A[0], ++A_smem,

tcgen05.cp tmem_A[1], ++A_smem,

tcgen05.cp tmem_A[2], ++A_smem,

tcgen05.cp tmem_A[3], ++A_smem,

tcgen05.mma tmem_accum, tmem_A[0], ++B_smem

tcgen05.mma tmem_accum, tmem_A[1], ++B_smem

tcgen05.mma tmem_accum, tmem_A[2], ++B_smem

tcgen05.mma tmem_accum, tmem_A[3], ++B_smem

tcgen05.commit ++mbar_smem_empty

Loop over
all the K iterations

tcgen05.commit mbar_tmem_full

More pipelined instruction descriptions in PTX ISA.

Blackwell NVFP4 Tensor Core

tcgen05.mma with block scaling factors, take NVFP4 as example

A&B data type: SE2M1

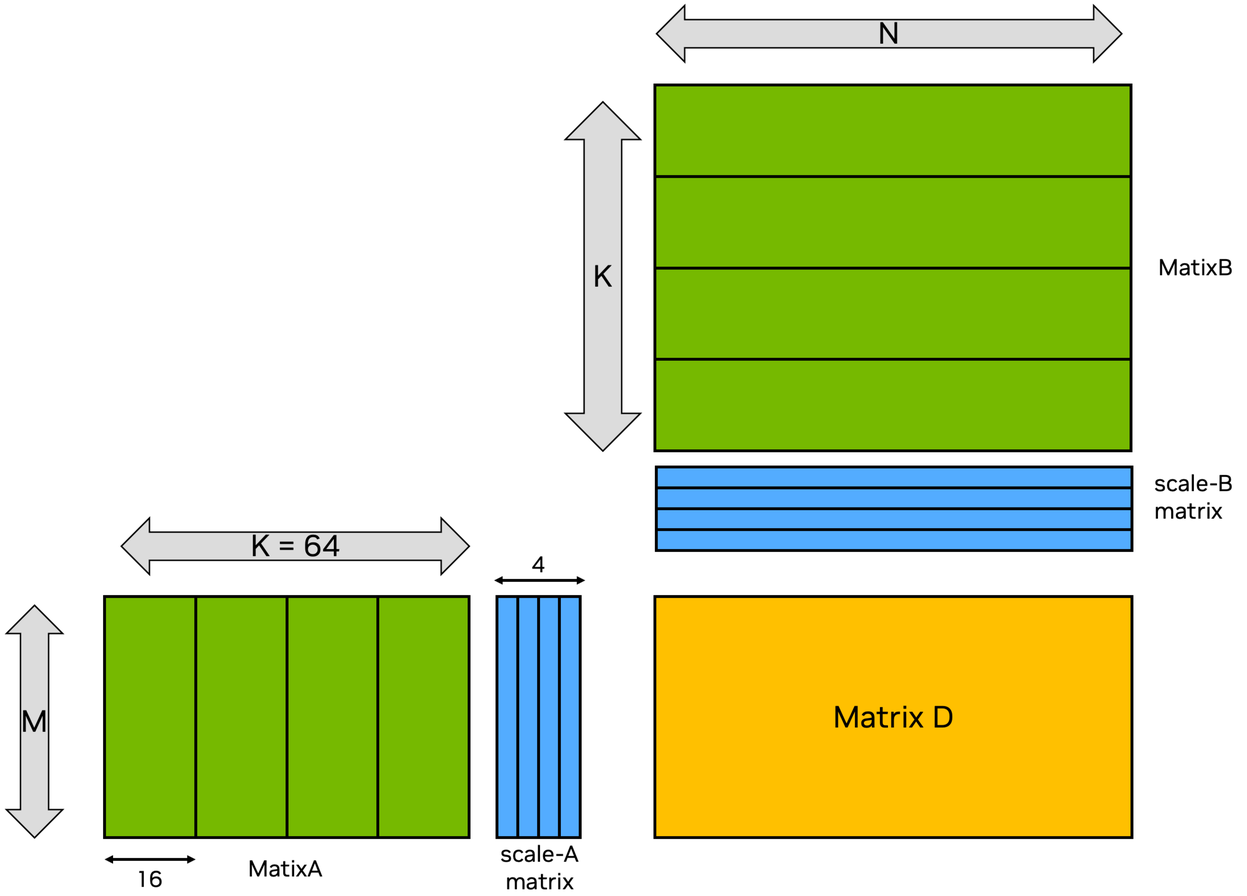
Operand A in TMEM or SMEM.

Operand B in SMEM

Scale factors data type: UE4M3

Operand A-sfs & B-sfs in TMEM

D[M][N] += (Asfs[M][0] x A[M][0:15]) x (Bsfs[0][N] x B[0:15][N])
+ (Asfs[M][1] x A[M][16:31]) x (Bsfs[1][N] x B[16:31][N])
+ (Asfs[M][2] x A[M][32:47]) x (Bsfs[2][N] x B[32:47][N])
+ (Asfs[M][3] x A[M][48:63]) x (Bsfs[3][N] x B[48:63][N])



Additional scale factors movement (SMEM->TMEM)is required.

Leader thread of the warp

All threads in the warp

Blackwell NVFP4 TensorCore pipeline

tcgen05.cp and tcgen05.mma are implicitly pipelined from same thread & same execution scope(1CTA or 2CTA)

A in SMEM, B in SMEM, Asfs in TMEM, Bsfs in TMEM

mbarrier.try_wait(++mbar_smem_full)

tcgen05.cp tmem_Asfs[0], ++Asfs_smem
tcgen05.cp tmem_Bsfs[0], ++Bsfs_smem
⋮
tcgen05.cp tmem_Asfs[3], ++Asfs_smem
tcgen05.cp tmem_Bsfs[3], ++Bsfs_smem

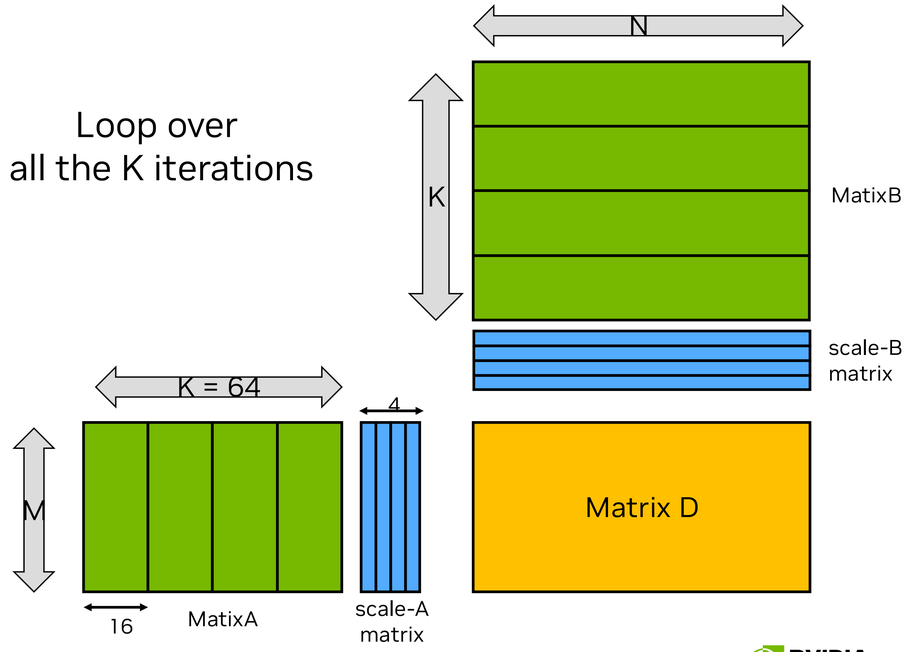
tcgen05.mma tmem_accum, ++A_smem, ++B_smem, tmem_Asfs[0], tmem_Bsfs[0]
tcgen05.mma tmem_accum, ++A_smem, ++B_smem, tmem_Asfs[1], tmem_Bsfs[1]
tcgen05.mma tmem_accum, ++A_smem, ++B_smem, tmem_Asfs[2], tmem_Bsfs[2]
tcgen05.mma tmem_accum, ++A_smem, ++B_smem, tmem_Asfs[3], tmem_Bsfs[3]

tcgen05.commit ++mbar_smem_empty

tcgen05.commit mbar_tmem_full

Loop over
all the K iterations

4x Tensor Core instructions for Ktile_128B

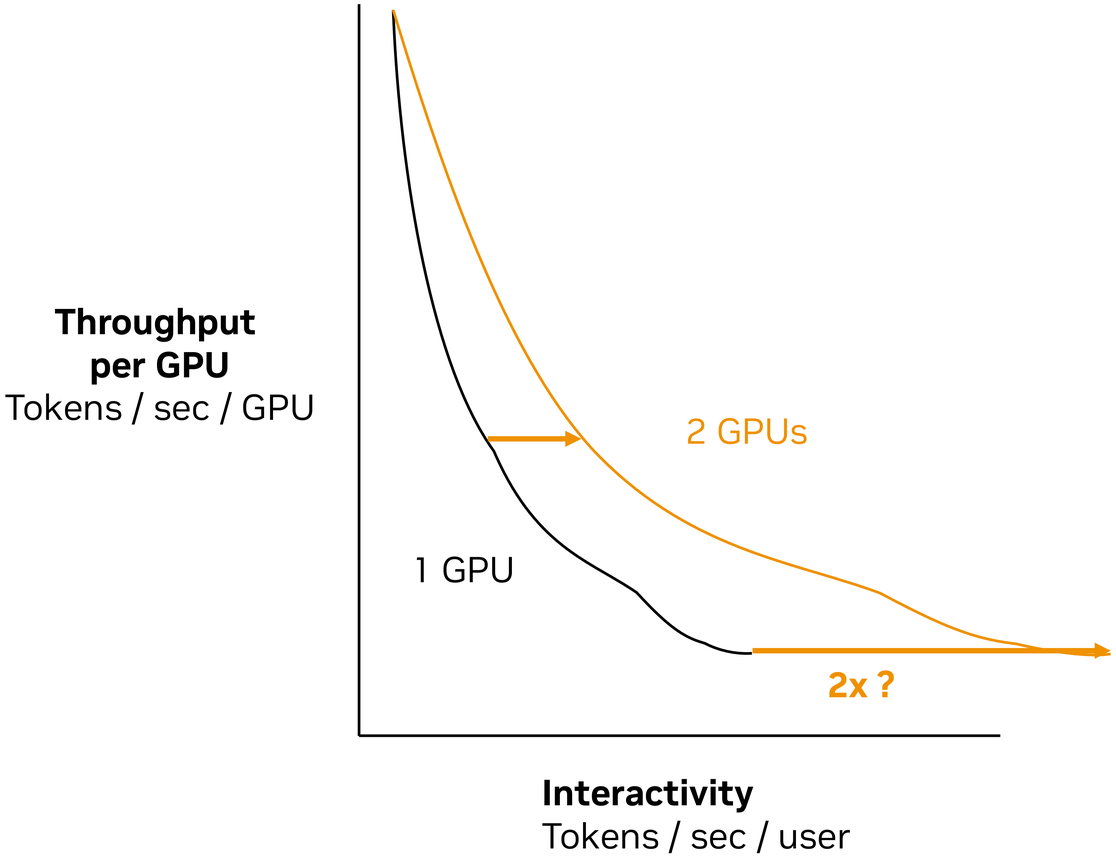


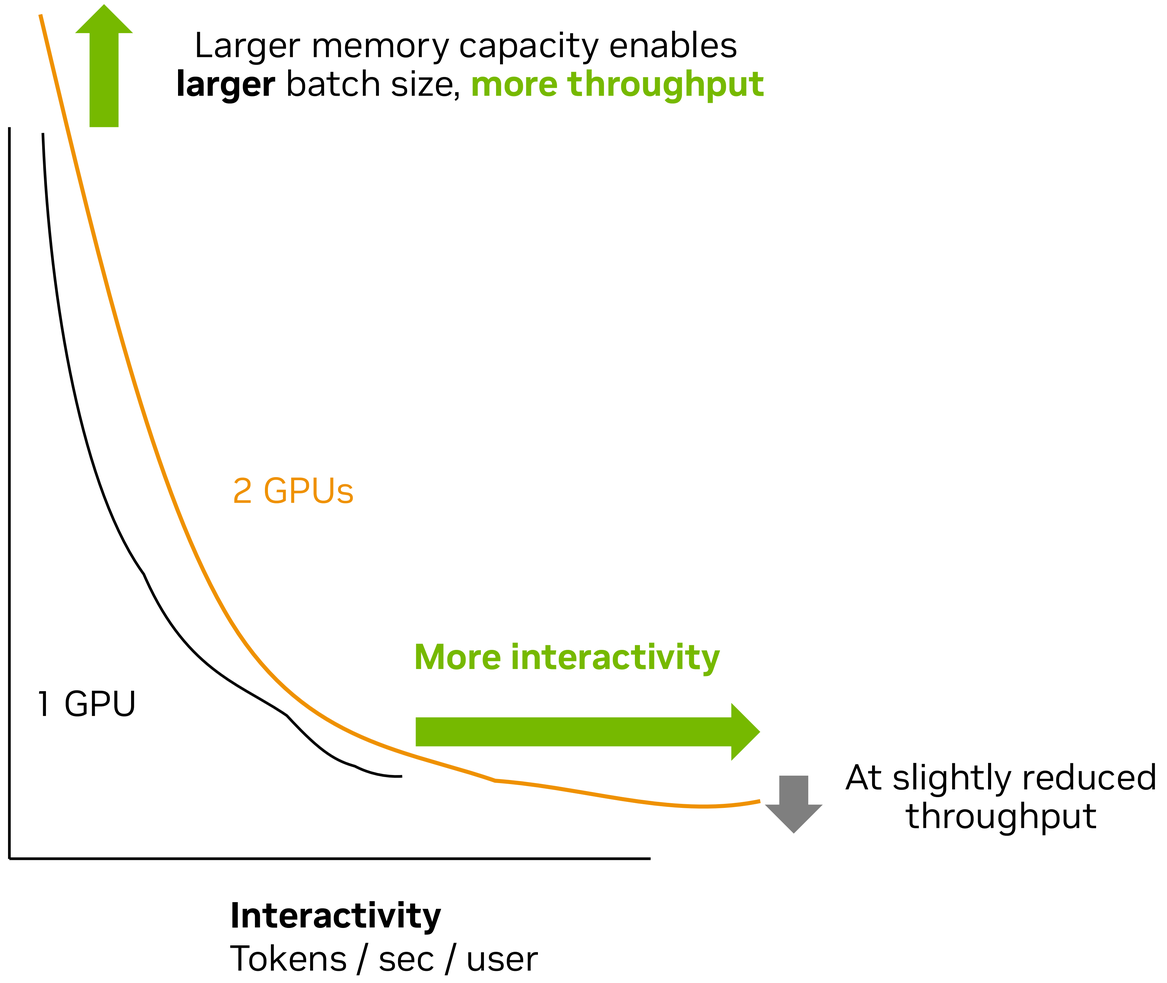
System-level optimizations

Using more GPUs

Effect on interactivity and throughput

Does doubling the GPUs double the interactivity?





Using more GPUs

Distributing the work across GPUs

Expert parallelism (EP)

Each GPU gets **half the experts**

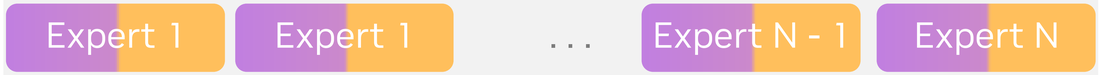
May suffer from expert imbalance

- One GPU could get all tokens – other GPU none

Less NVLink communication

- Communicates 1 token per expert token

Tensor parallelism (TP)



Each GPU gets **half of each expert**

Always balanced

- Each GPU processes same number of tokens

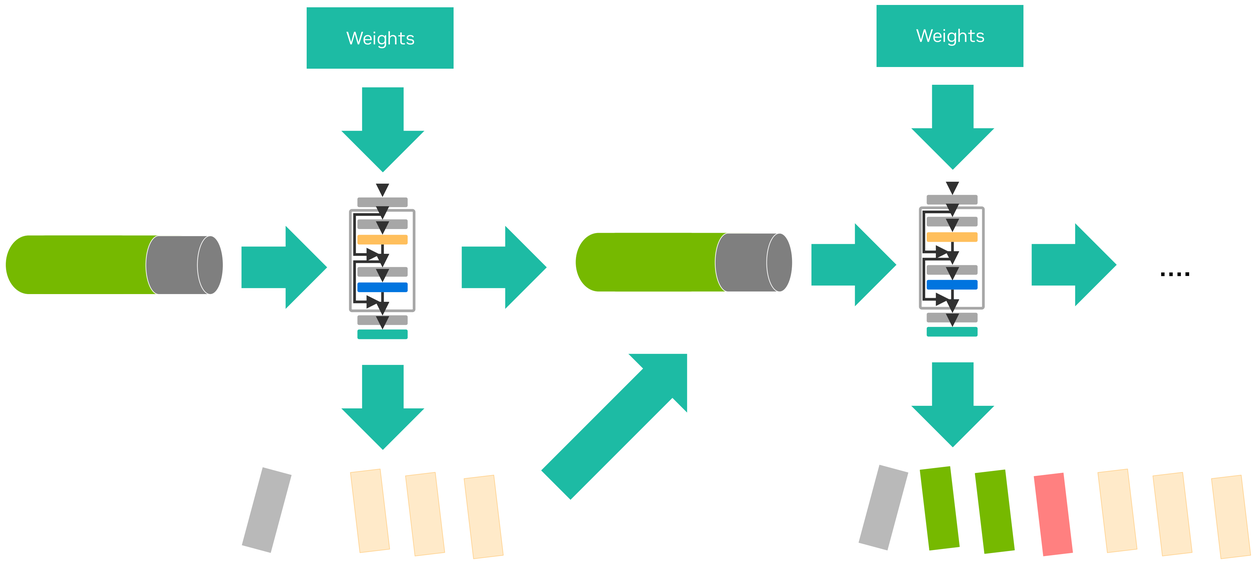
More NVLink communication

- Communicates 2 tokens per expert output token

More suited to **throughput** regime

More suited to **latency** regime

Multi-token prediction / Speculative decoding



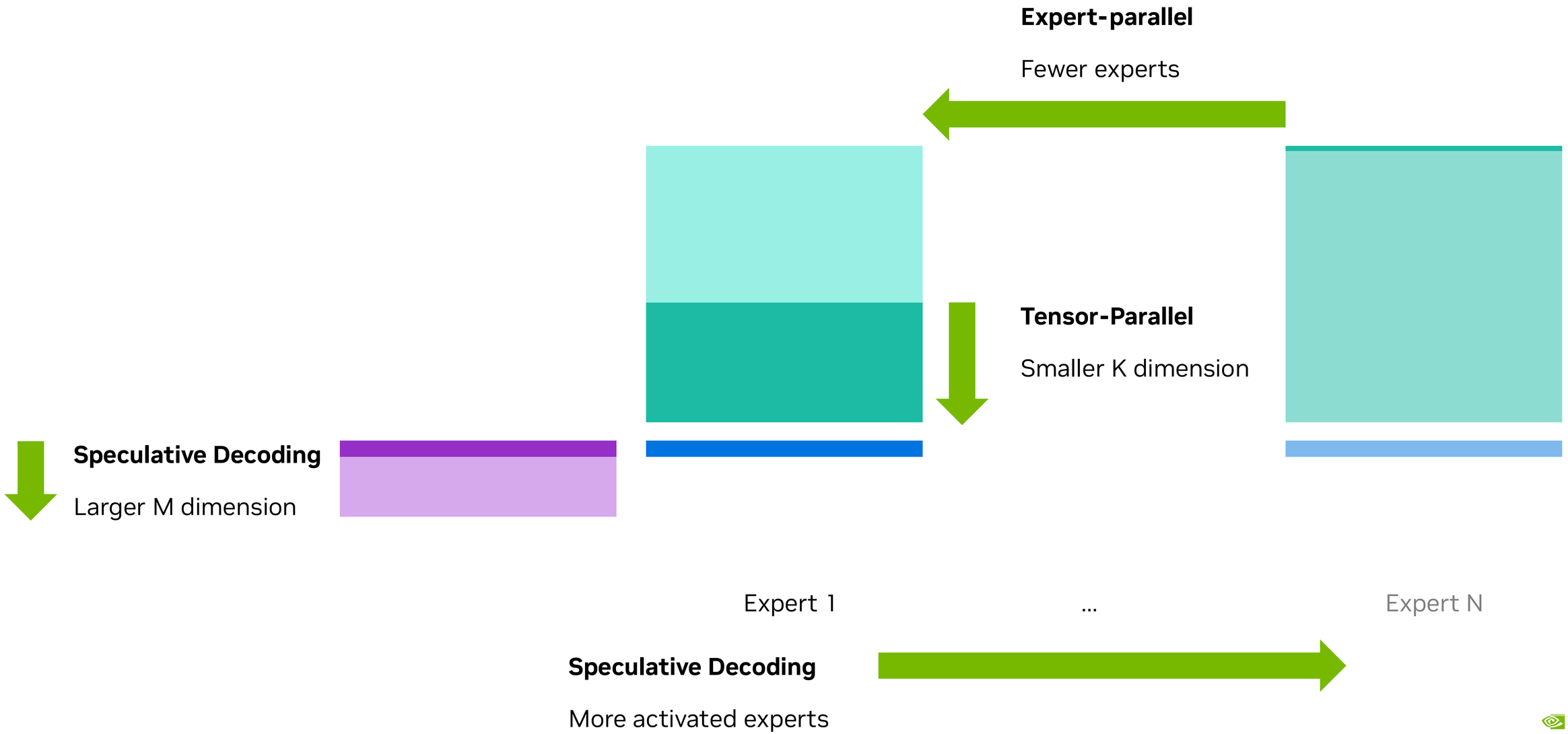
- Generate one token and **several speculative tokens**
- Speculative tokens can be be generated by
  - A dedicated smaller model running parallel
  - Extra network layers in the main LLM
- The extra tokens must be **validated**

Implications

- The batch size for the Mixture-of-Experts is higher
- The model can generate 1-4 tokens per iteration

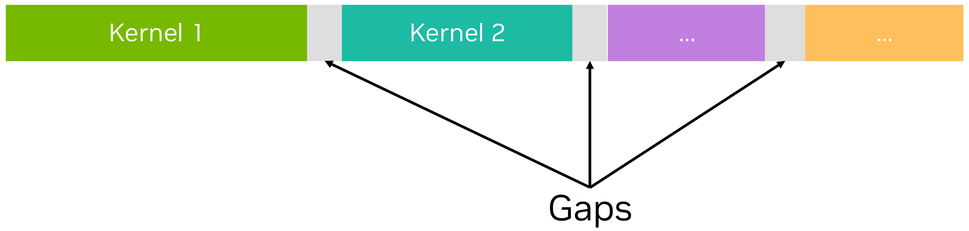
Impact of system-level optimizations on MoE kernel

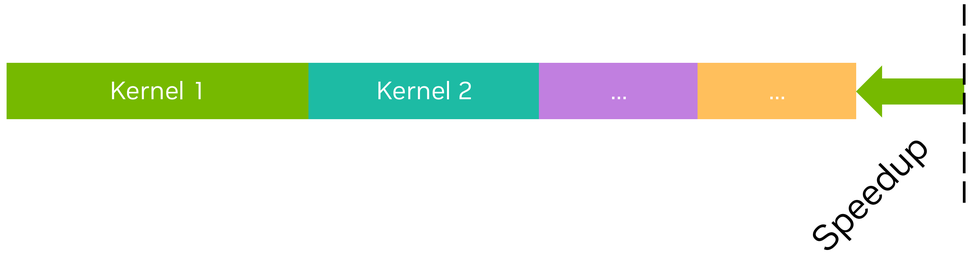
Summary



System-level optimizations for short-running kernels

CUDA Graph





Normal kernel launches

- Some launch latency between kernels
- Decide kernel parameters at launch time

CUDA Graph launches

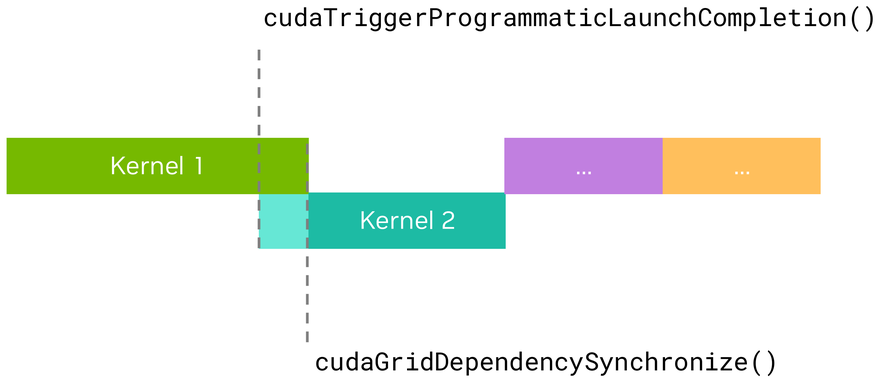
- Less launch latency
- Create graph once
  - Kernel parameters are typically fixed* at creation time
  - Grid size is fixed at creation time
    - Accommodate dynamic # of activated experts in the kernel
- Launch graph multiple times

For more information: S72686 CUDA Techniques to Maximize Concurrency and System Utilization

* APIs exist to update parameters – cannot update on the fly, however

System-level optimizations for short-running kernels

Programmatic Dependent Launch (PDL)



Normal kernel launches

- Kernels in same stream are launched back-to-back

PDL Launch

- Kernels in same stream may overlap
- Launch subsequent grid earlier using
  - `cudaTriggerProgrammaticLaunchCompletion()`
- Wait for data from previous grid using
  - `cudaGridDependencySynchronize()`
- Allows independent work to overlap
- Can give speedups even without independent work

For more information: S72686 CUDA Techniques to Maximize Concurrency and System Utilization

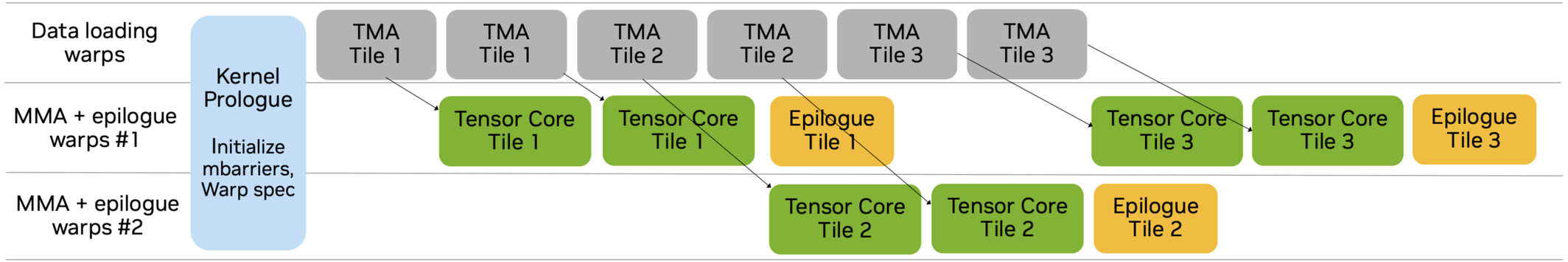
Warp specialized kernel design

Overall Warp-Specialized Kernel Design

Evolution Hopper to Blackwell

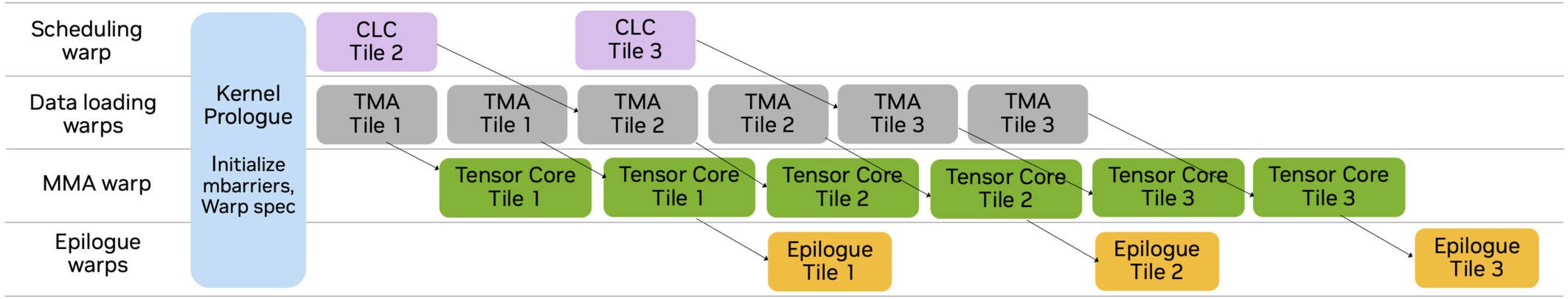
Hopper Ping-pong Warp Specialization

Acc. in RMEM → Epilogue and MMA on same threads → ping-pong between two MMA groups to overlap epilogue overhead



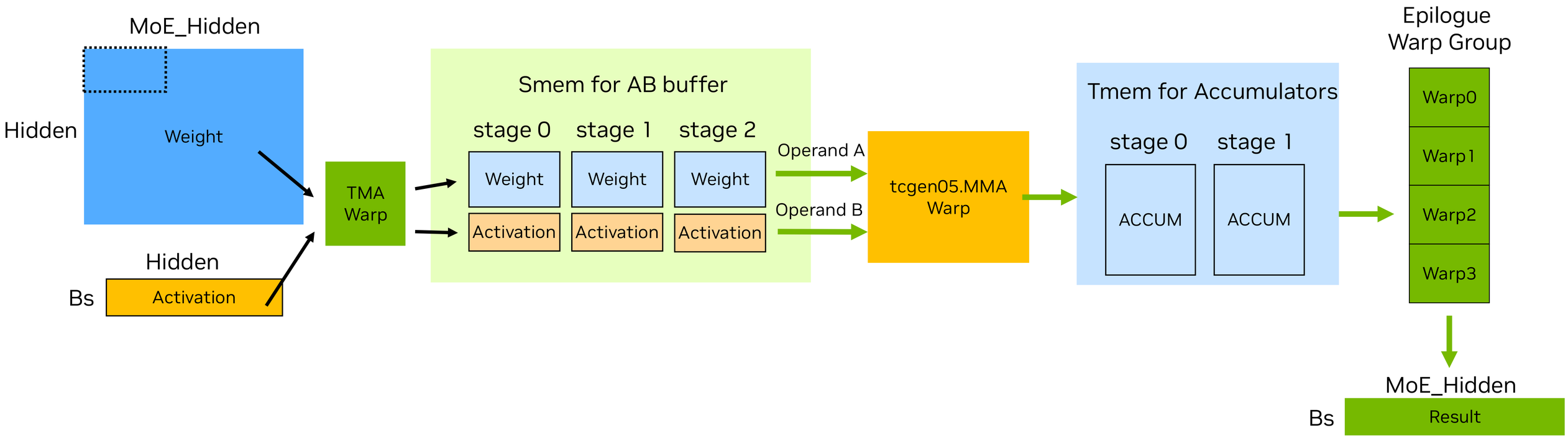
Blackwell Warp Specialization

Acc. in TMEM → Epilogue and MMA on separate threads → Two MMA threads not needed to overlap epilogue



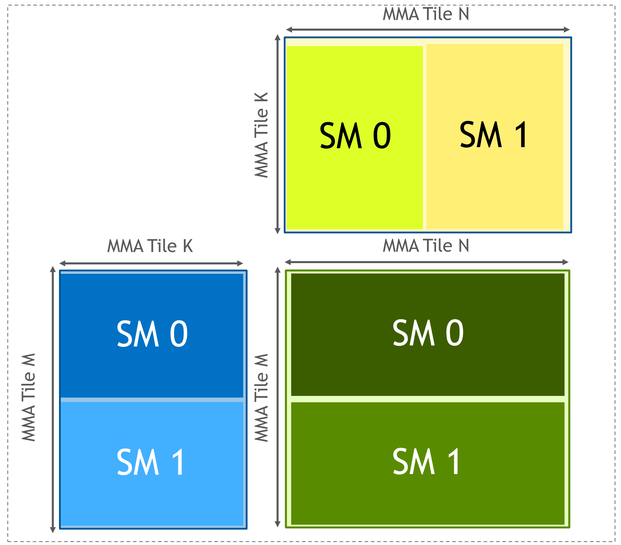
Blackwell Warp-Specialized Kernel Design

Core data movement



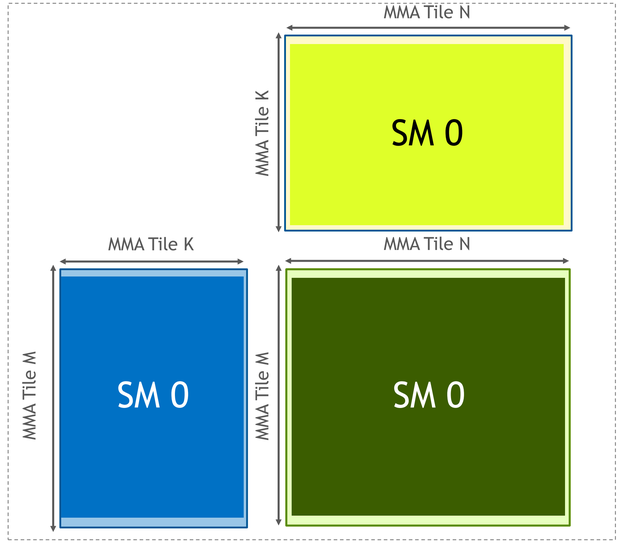
Tensor Core Warp

When to use 1CTA versus 2CTA?



- 2CTA mode
  - ✓ Better for large N
  - X More synchronization in MMA mainloop
    - ➢ But synchronization can be hidden
  - ✓ Smaller B in shared memory
    - ✓ More stages in shared memory
    - ✓ Can hide more memory latency

Better suited to **throughput regime**

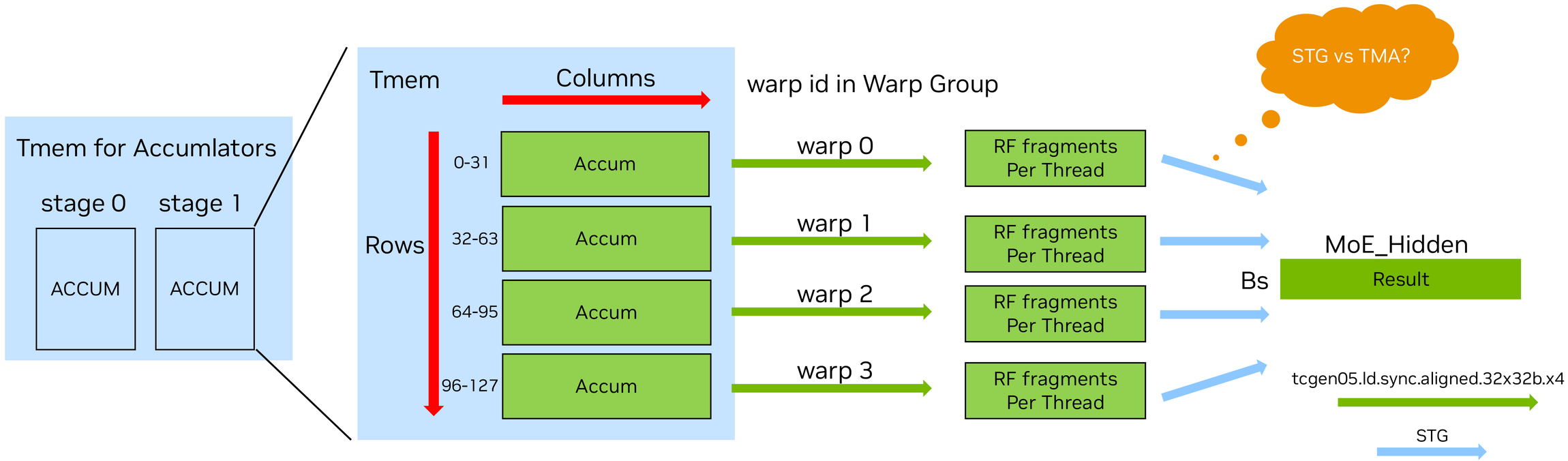


- 1CTA mode
  - ✓ Better for small N
  - ✓ Less synchronization in MMA mainloop
  - X Duplicates B in shared memory
    - ➢ **Activation is relatively small in low-latency**

Better suited to **latency regime**

Epilogue

4 warps access TMEM across all sub-partitions



TMA

- ✗ Requires additional shared memory
- ✗ Extra shared memory store + synchronization latency
- ✓ Save instruction issue bandwidth
- ✓ Supports any output layout (M, N-major)

STG (thread store to global)

- ✓ Low-overhead direct register to memory stores
- ✓ Avoids Smem + synchronization latency
- ✗ Lots of instructions when output is large
- ✗ Requires care to ensure memory coalescing

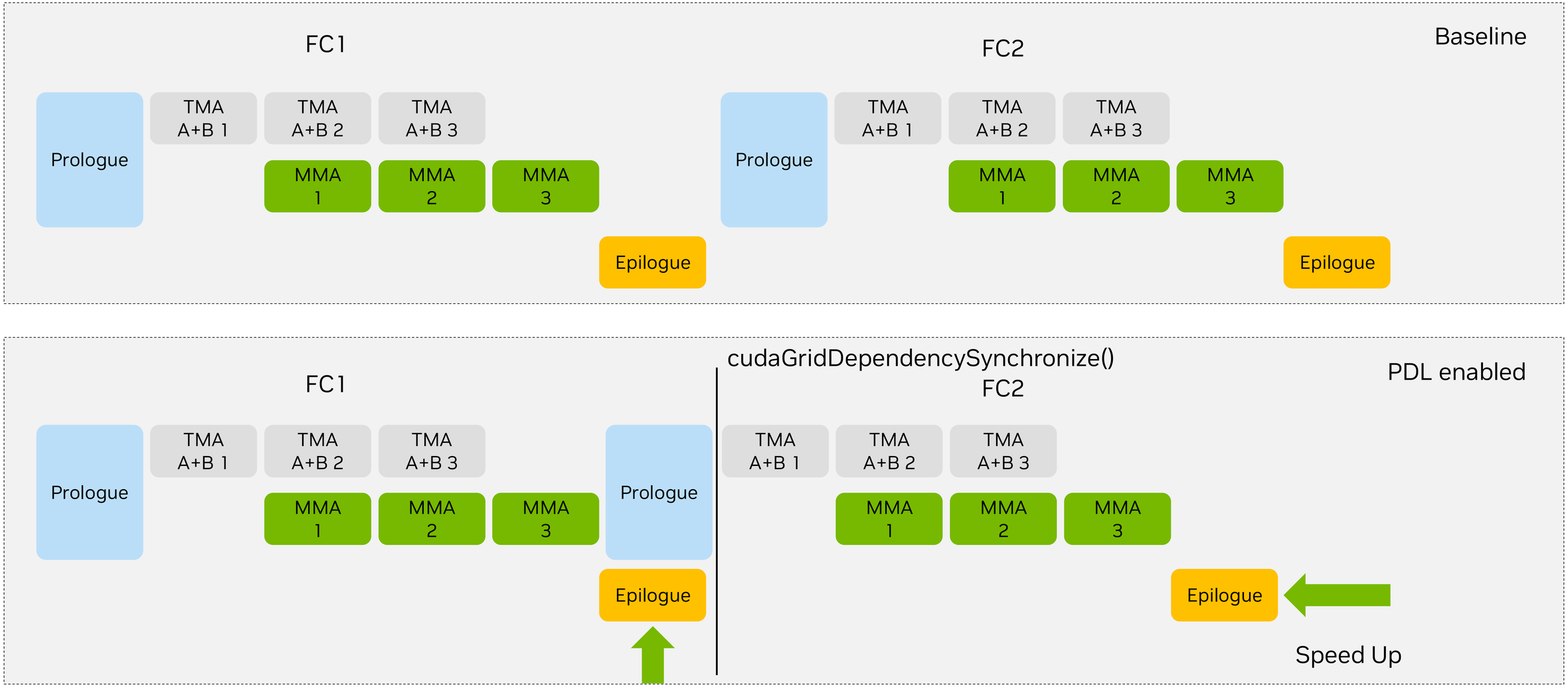
Better suited to **throughput regime** ⟷ Better suited to **latency regime**

NVIDIA

Latency-optimized Warp-Specialization

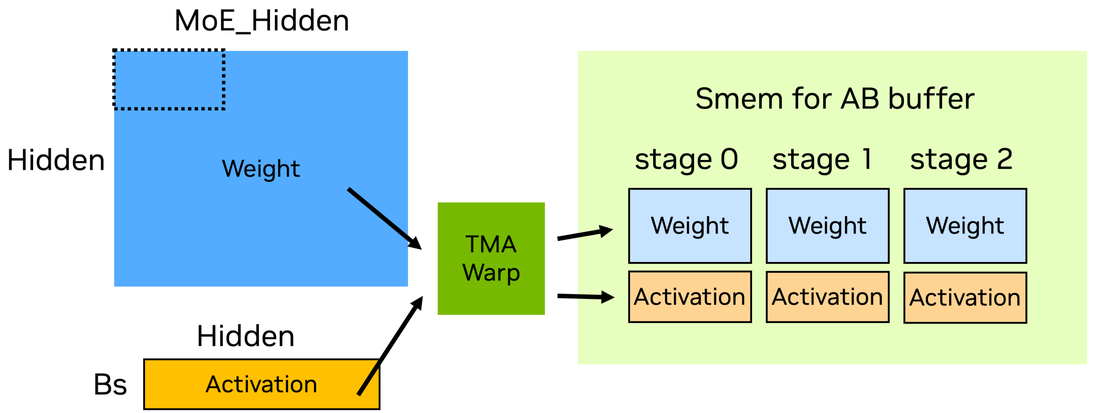
Latency-optimized Warp-Specialization

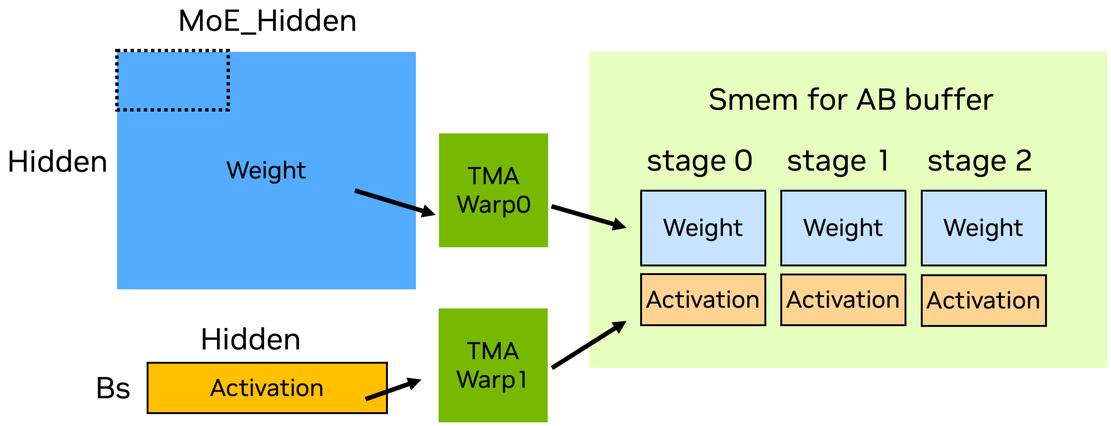
Using PDL to overlap prologue



TMA Warp(s) in Latency-optimized Warp-Specialization

1 TMA warp vs 2 TMA warps





Single TMA warp

- Loads of A and B must happen at the same time
- Fewer barrier updates

Two TMA warps

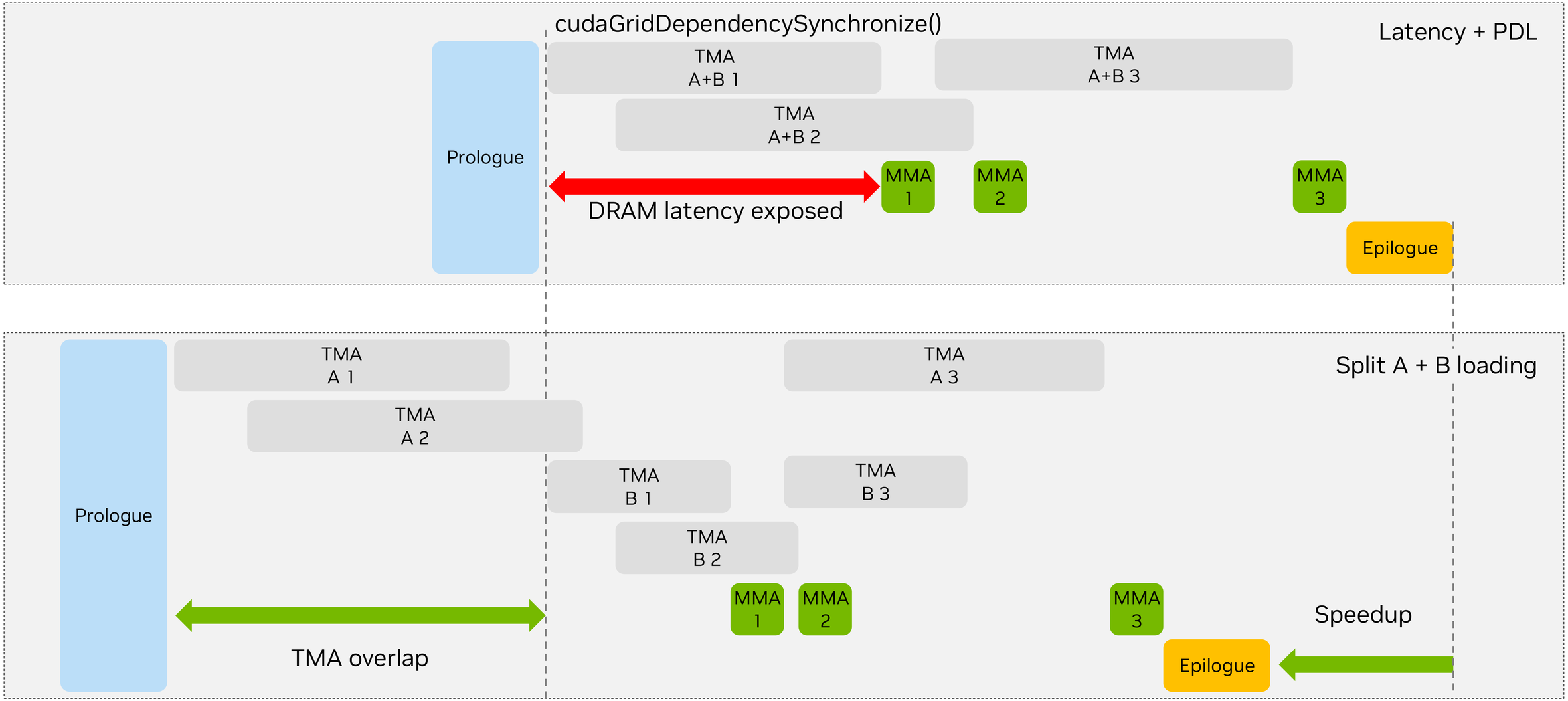
- Loads of A and B are decoupled
  - One warp won't block the other from issuing TMA requests.
  - With PDL, one warp can start loading early
- More barrier updates

More suited to **throughput** regime

More suited to **latency** regime

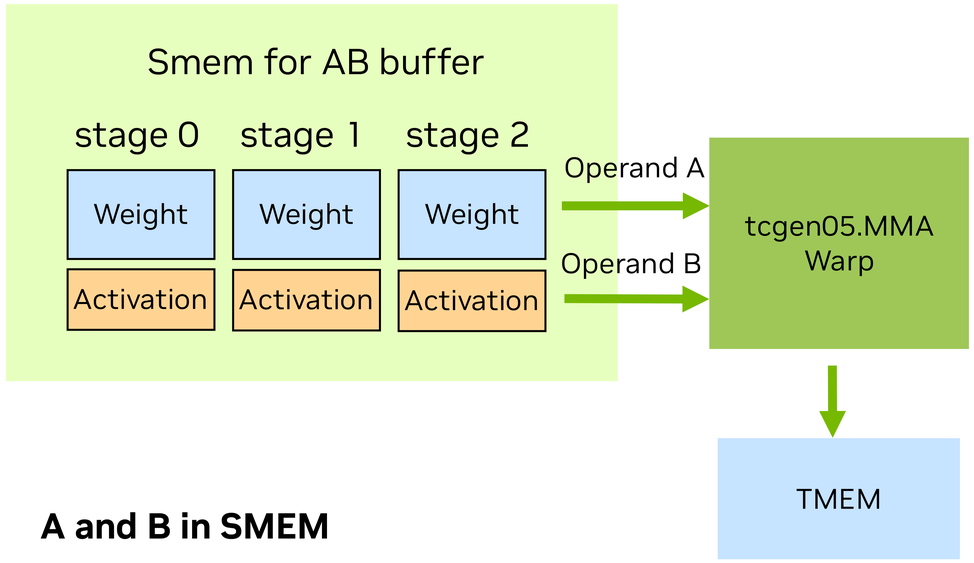
Latency-optimized Warp-Specialization

Overlap weights loading



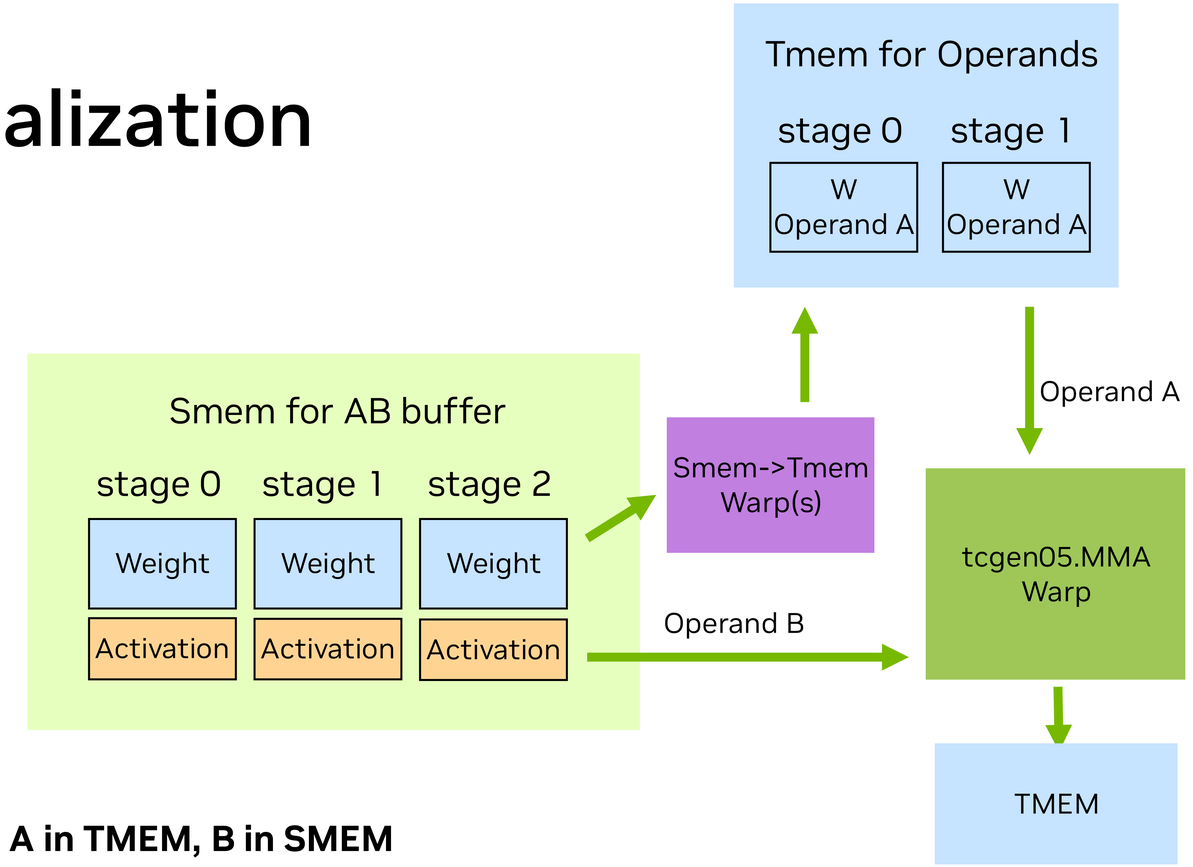
Latency-optimized Warp-Specialization

A in TMEM



A and B in SMEM

- Fewer data copies
- Fewer warps
- ✕ Fewer A stages



A in TMEM, B in SMEM

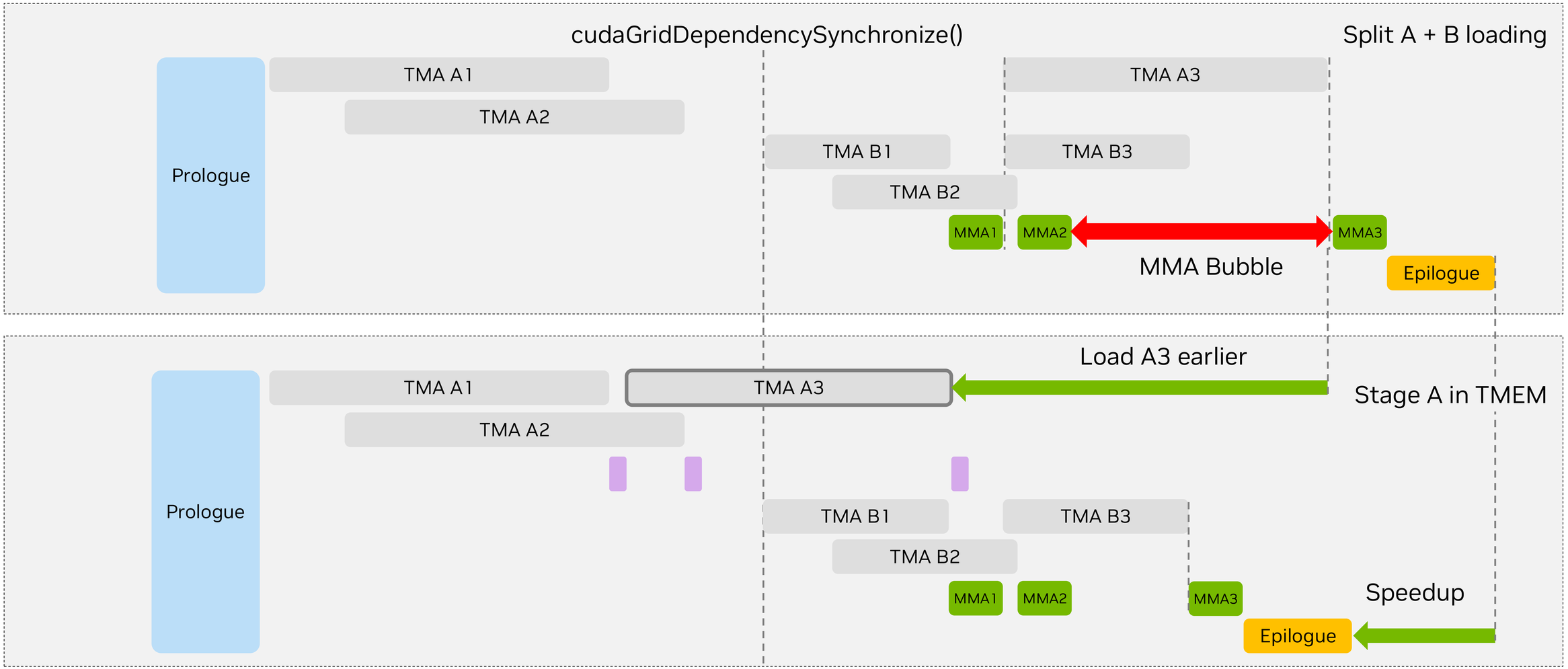
- ✕ More data copies (SMEM -> TMEM)
- ✕ More warps
- ✕ More TMEM usage
- More A stages – Higer tolerance for DRAM Latency
- Can refill SMEM quicker – less TMA bubbles

More suited to **throughput** regime

More suited to **latency** regime

Latency-optimized Warp-Specialization

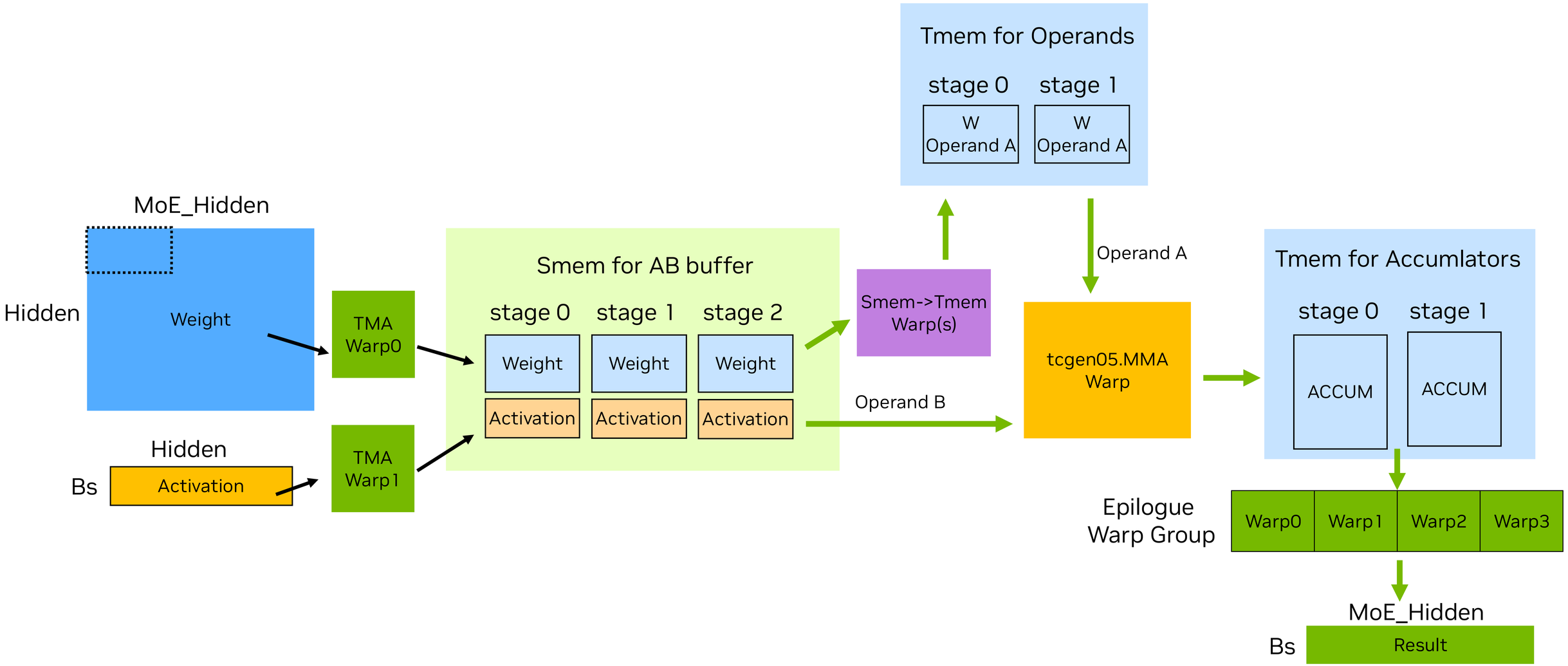
Add TMEM staging



Smem -> Tmem

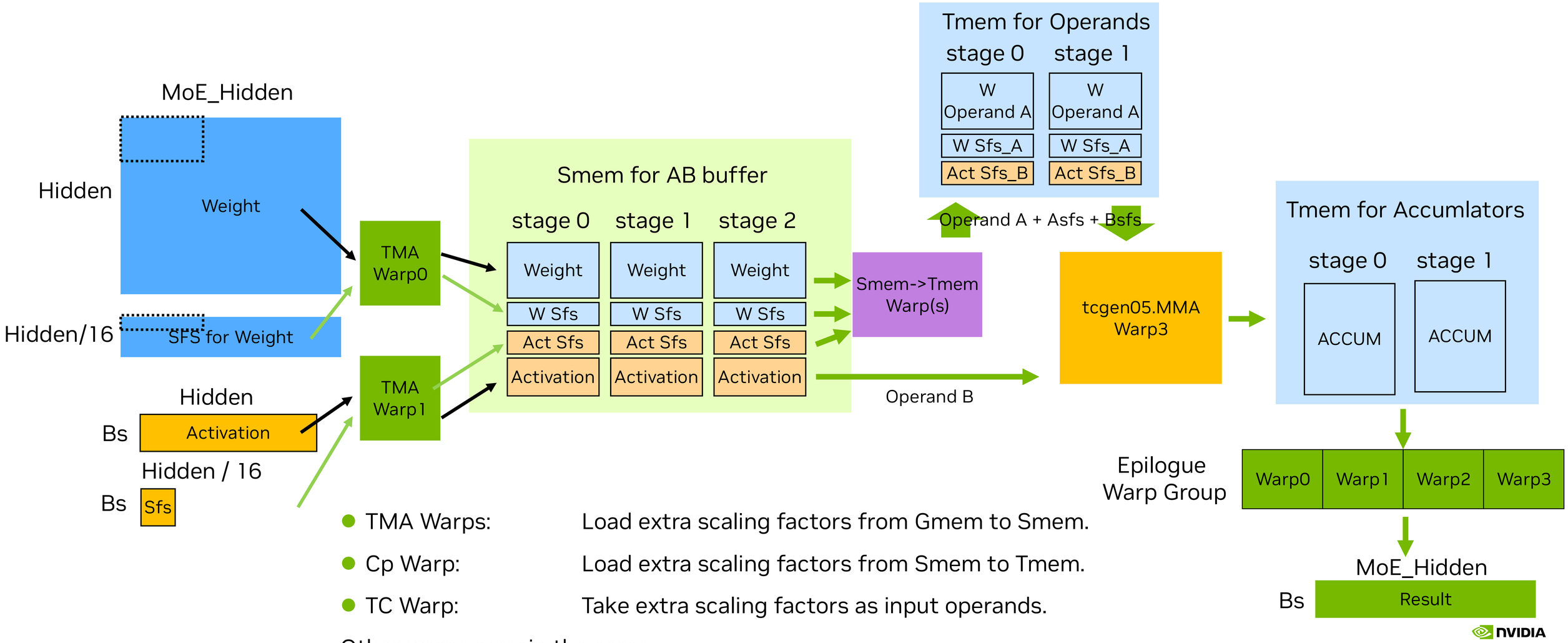
Latency-optimized Warp-Specialization

Key data movement



Latency-optimized Warp-Specialization

Scaling factors key data movment



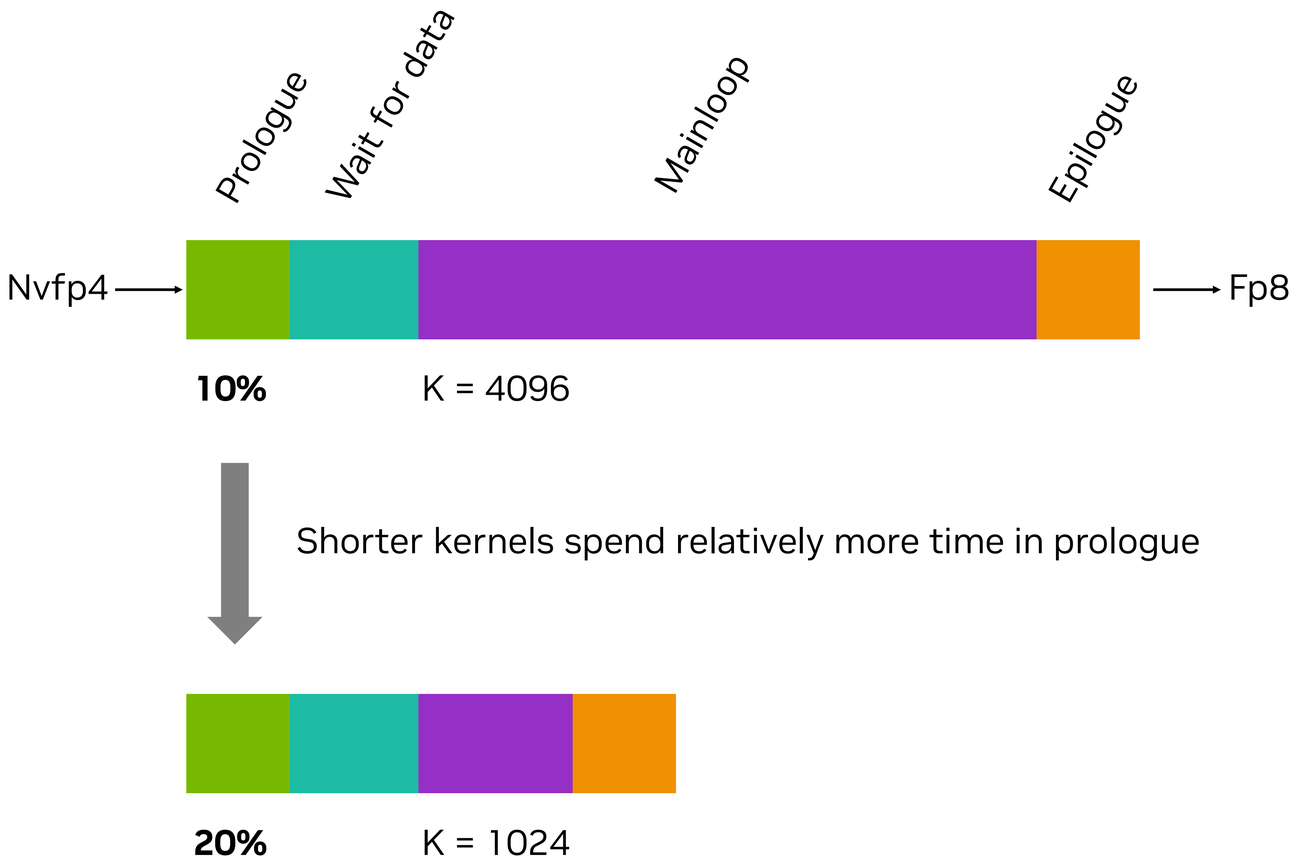
- TMA Warps: Load extra scaling factors from Gmem to Smem.
- Cp Warp: Load extra scaling factors from Smem to Tmem.
- TC Warp: Take extra scaling factors as input operands.

Other warps remain the same.

Prologue

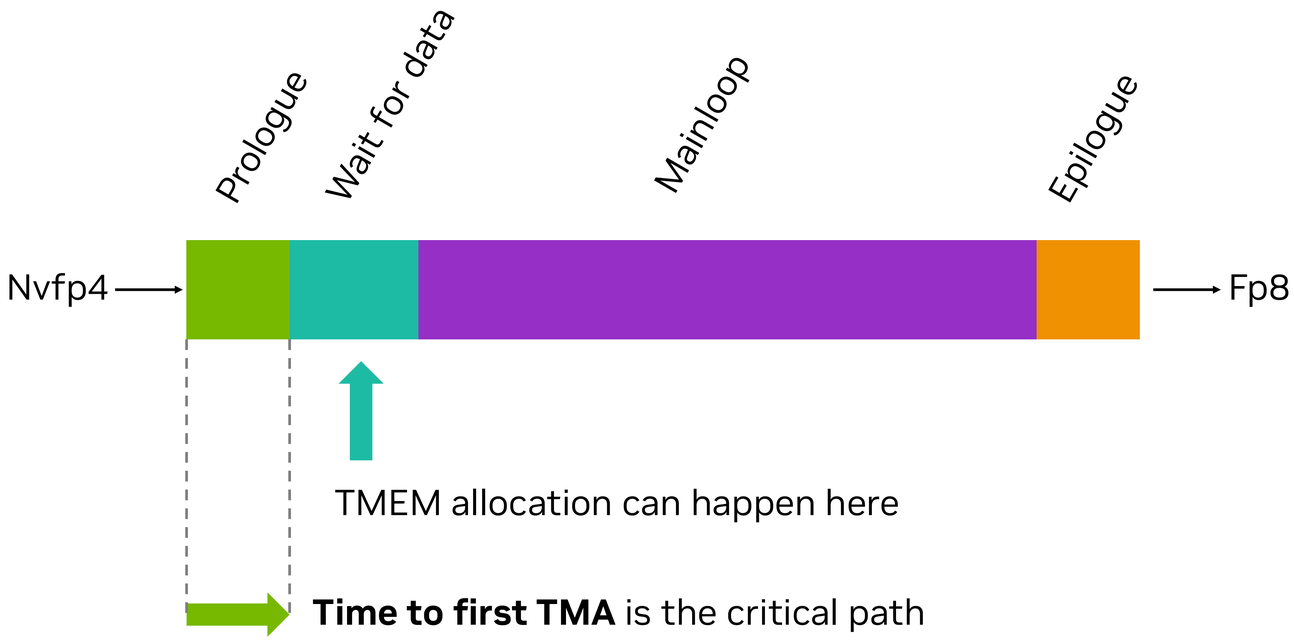
Prologue

- What do we need to start the mainloop?
  - Initialize shared memory barriers
  - Sync cluster (ensure visibility of shared memory barriers)
  - Issue TMA to fetch A and B
  - Allocate TMEM



Prologue

- What do we need to start the mainloop?
  - Initialize shared memory barriers
  - Sync cluster (ensure visibility of shared memory barriers)
  - Issue TMA to fetch A and B
  - Allocate TMEM



Prologue

Faster barrier initialization

Serialized mbarrier initialization

- **One** thread initializes **multiple** barriers

```
for (int i = 0; i < num_barriers; ++i) {
  ptx::mbarrier_init(barrier_array + i, arrival_count);
}
```

Hopper

- Requires multiple instructions

Vectorized mbarrier initialization

- **Several** threads initialize a **single** barrier

```
if (threadIdx.x < num_barriers) {
  ptx::mbarrier_init(barrier_array + threadIdx.x, arrival_count);
}
```

New in Blackwell

- Initialize multiple barriers **in a single instruction**

Prologue

Faster cluster sync time

```
// Initialize barriers
if (threadIdx.x < num_barriers) {
  ptx::mbarrier_init(barrier_array + threadIdx.x, arrival_count);
}

// Make barrier visible


ptx::barrier_cluster_arrive(ptx::sem_release);
ptx::barrier_cluster_wait(ptx::sem_acquire);
```

```
// Initialize barriers
if (threadIdx.x < num_barriers) {
  ptx::mbarrier_init(barrier_array + threadIdx.x, arrival_count);
}

// Make barrier visible
ptx::fence_mbarrier_init(ptx::sem_release, ptx::scope_cluster);

ptx::barrier_cluster_arrive(ptx::sem_relaxed);
ptx::barrier_cluster_wait(ptx::sem_acquire);
```

Slow  Fast – Always preferred

For more info: S72683 - CUDA Techniques to Maximize Memory Bandwidth and Hide Latency

Data Loading

TMA runtime dynamic box size

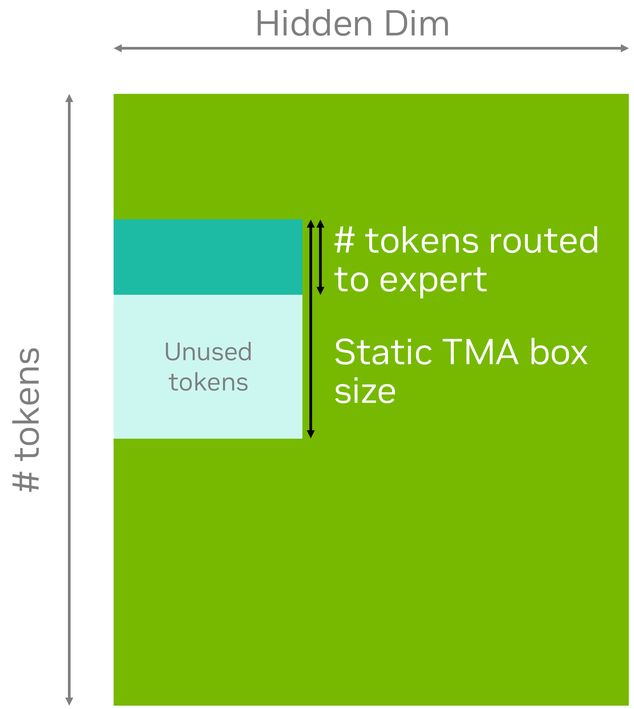
Problem statement

TMA box size is **static**

- When we use TMA, we create a tensor map on the host before launching the kernel
- The tensor map determines the box size
- The tensor map cannot easily be changed

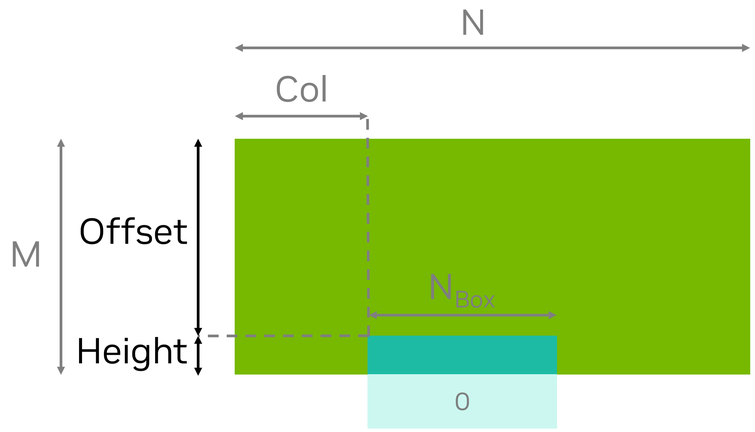
However: the number of tokens routed to an expert is **dynamic** at runtime

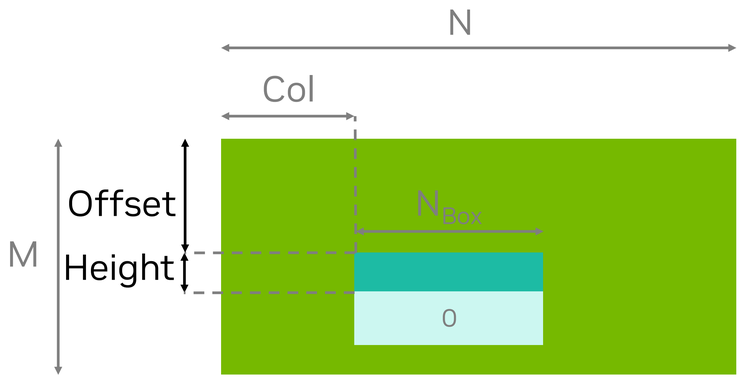
If we use a large static TMA box size, we are **loading unused tokens**



TMA runtime dynamic box size

What we want





What we can already do

- Load a box that is slightly over the edge
- Outside values get filled with zero

What we want to do

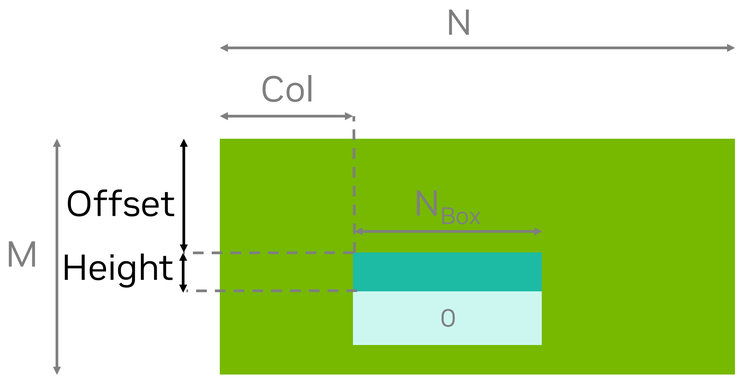
- Load a box at any offset
- With a dynamic size
- Rows beyond beyond height are filled with zero

TMA runtime dynamic box size

A peek at the solution

To get a **2-dimensional box** of Height x N_{Box} out of an M x N matrix:

- Create a **4-dimensional tensor map** with
  - `Size` = (N, M, 2^{34}, 2^{34})
  - `Stride` = (2^{34} − N, N, N, 1)
  - `Box size` = (1, 1, 256, N_{Box})
- Use coordinates:
  - `Coordinates` = (Col, 2^{30} − Height, Offset + Height, 2^{30})



Caveats

- Backing memory must be **at least 128KB(!)**
  - Otherwise: potential for page faults
- `Offset + height` must be less than `M`
  - No out-of-bounds checking by TMA anymore(!)

Thanks to OpenAI for finding and sharing this trick!

TMA runtime dynamic box size

Solve for coordinates, size, stride

Mission accomplished

Coordinates	Tensor size	Box size	Effective box size
??	??	1	1
??	??	1	1
M - Height	M	Box_M	min(Height, M_{Box})
Col	N	Box_N	max(0, min(N - Col, N_{Box}))

Coordinates	Stride	Pointer offset contribution
??	??	
??	??	
M - Height	A	M * A – Height * A
Col	1	Col
Total		**Col + M * A – Height * A**

Goal: Col + Offset * N

Constraints:

- We cannot have negative coordinates
- We cannot have negative strides

TMA runtime dynamic box size

Solve for coordinates, size, stride

Coordinates	Tensor size	Box size	Effective box size
??	??	1	1
Offset + Height	??	1	1
M - Height	M	Box_M	min(Height, M_{Box})
Col	N	Box_N	max(0, min(N - Col, N_{Box}))

Coordinates	Stride	Pointer offset contribution
??	??	
Offset + Height	N	+ Offset * N + Height * N
M - Height	A	M * A − Height * A
Col	1	Col
Total		**Col + Offset * N + M * A − Height * (N − A)**

- Add Offset * N

TMA runtime dynamic box size

Solve for coordinates, size, stride

Coordinates	Tensor size	Box size	Effective box size
??	??	1	1
Offset + Height	??	1	1
M - Height	M	Box_M	min(Height, M_{Box})
Col	N	Box_N	max(0, min(N - Col, N_{Box}))

Coordinates	Stride	Pointer offset contribution
??	??	
Offset + Height	N	+ Offset * N + Height * N
M - Height	**N**	M * N − Height * **N**
Col	1	Col
Total		**Col + Offset * N + M * N − 0**

- Add Offset * N
- Correct Height * (N − A)

TMA runtime dynamic box size

Solve for coordinates, size, stride

Coordinates	Tensor size	Box size	Effective box size
M	??	1	1
Offset + Height	??	1	1
M - Height	M	Box_M	min(Height, M_{Box})
Col	N	Box_N	max(0, min(N - Col, N_{Box}))

Coordinates	Stride	Pointer offset contribution
M	X - N	- M * N + M * X
Offset + Height	N	+ Offset * N + Height * N
M - Height	N	M * N - Height * N
Col	1	Col
Total		**Col + Offset * N + 0 - 0 + M * X**

- Add Offset * N
- Correct Height * (N - A)
- Correct M * N

TMA runtime dynamic box size

Solve for coordinates, size, stride

Coordinates	Tensor size	Box size	Effective box size
2^{30}	??	1	1
Offset + Height	??	1	1
M - Height	2^{30}	Box_M	min(Height, M_{Box})
Col	N	Box_N	max(0, min(N - Col, N_{Box}))

Coordinates	Stride	Pointer offset contribution
2^{30}	2^{34} - N	- 2^{30} * N + 2^{30} * 2^{34}
Offset + Height	N	+ Offset * N + Height * N
2^{30} - Height	N	2^{30} * N − Height * N
Col	1	Col
Total		**Col + Offset * N + 0 − 0 + 0**

- Add Offset * N
- Correct Height * (N − A)
- Correct M * N
- Correct M * X: M = 2^{30}, X = 2^{34}

TMA runtime dynamic box size

Solve for coordinates, size, stride

Coordinates	Tensor size	Box size	Effective box size
2^{30}	2^{34}	1	1
Offset + Height	2^{34}	1	1 (if offset + size < 2^{34})
M - Height	M	Box_M	min(Height, M_{Box})
Col	N	Box_N	max(0, min(N - Col, N_{Box}))

Coordinates	Stride	Pointer offset contribution
2^{30}	2^{34} - N	- 2^{30} * N + 2^{30} * 2^{34}
Offset + Height	N	+ Offset * N + Height * N
2^{30} - Height	N	2^{30} * N − Height * N
Col	1	Col
Total		**Col + Offset * N + 0 − 0 + 0**

- Add Offset * N
- Correct Height * (N − A)
- Correct M * N
- Correct M * X: M = 2^{30}, X = 2^{34}
- Fill in sizes

Constraints are met:

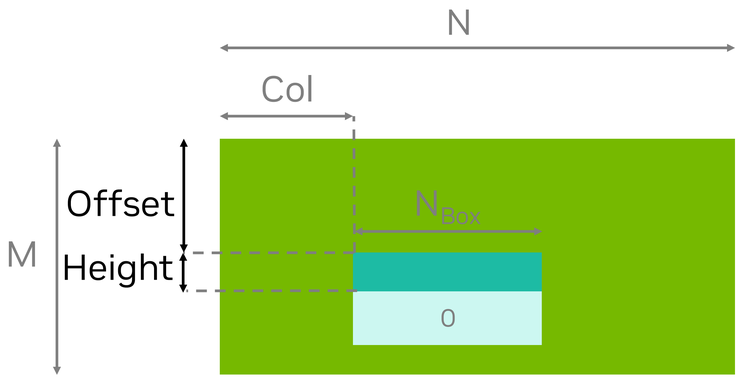
- All coordinates are positive
- All strides are positive (if N < 2^{34})

TMA runtime dynamic box size

The recipe

To get a box of Height x N_{Box} out of an M x N matrix:

- Create a 4-dimensional tensor map with
  - `Size` = (N, M, 2^{34}, 2^{34})
  - `Stride` = (2^{34} − N, N, N, 1)
  - `Box size` = (1, 1, 256, N_{Box})
- Use coordinates:
  - `Coordinates` = (Col, 2^{30} − Height, Offset + Height, 2^{30})



Caveats

- Backing memory must be **at least 128KB(!)**
  - Otherwise: potential for page faults
- `Offset + height` must be less than `M`
  - No out-of-bounds checking by TMA anymore(!)

Thanks to OpenAI for finding and sharing this trick!

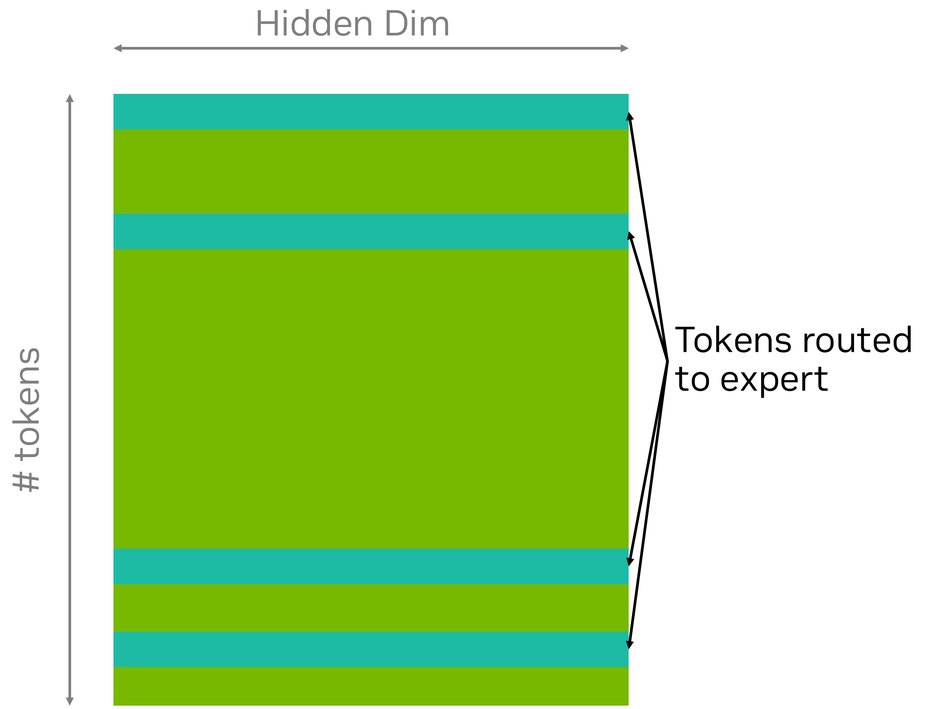
TMA GATHER4

Problem statement

TMA normally loads **contiguous** boxes of memory

However, the tokens routed to an expert are **non-contiguous**

We **cannot** use TMA with a **large box size**



TMA GATHER4

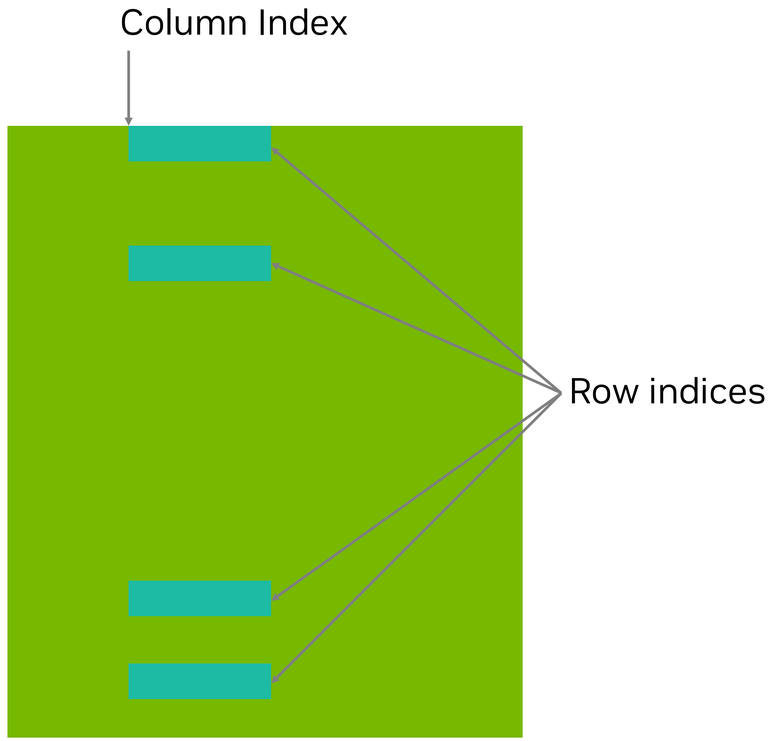
Problem statement

PTX Instruction

`cp.async.bulk.tensor.2d.tile::gather4.shared::cta.global`

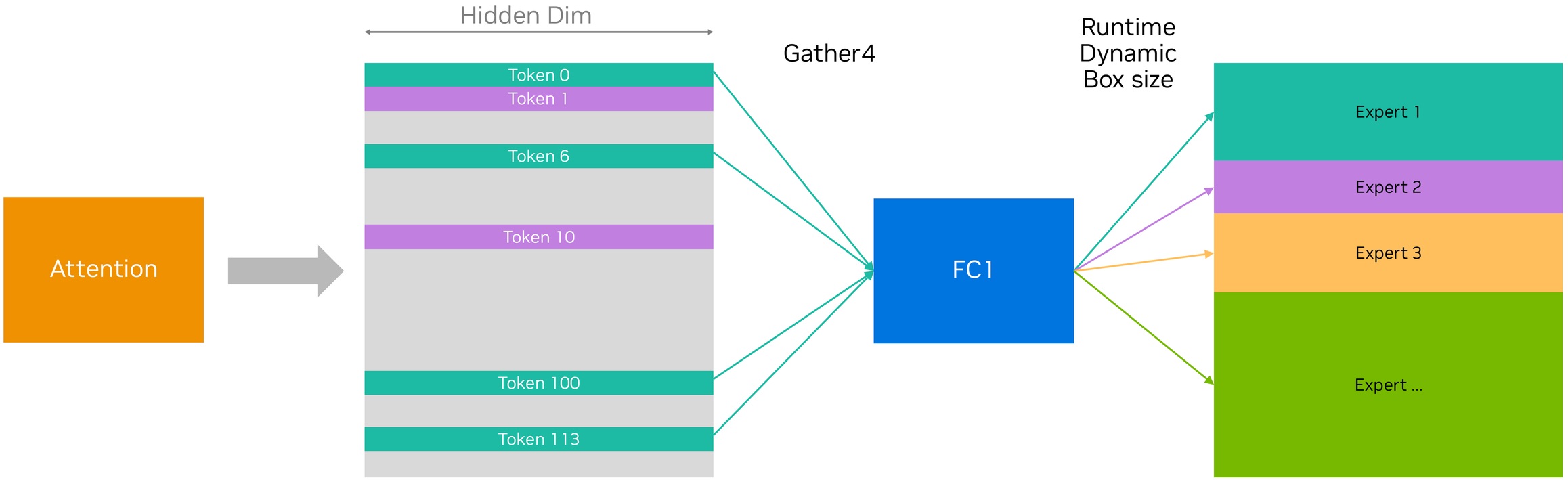
- Takes 1 column index and **4 row indices**
- Issue is per-warp, like other cp.async.bulk instructions
- Swizzle{32,64,128, etc} is supported
  - With Swizzle128, reads 4 x 128 = 512B per instruction

- Prefer to **use 2-4 warps** instead of 1 to keep TMA unit busy



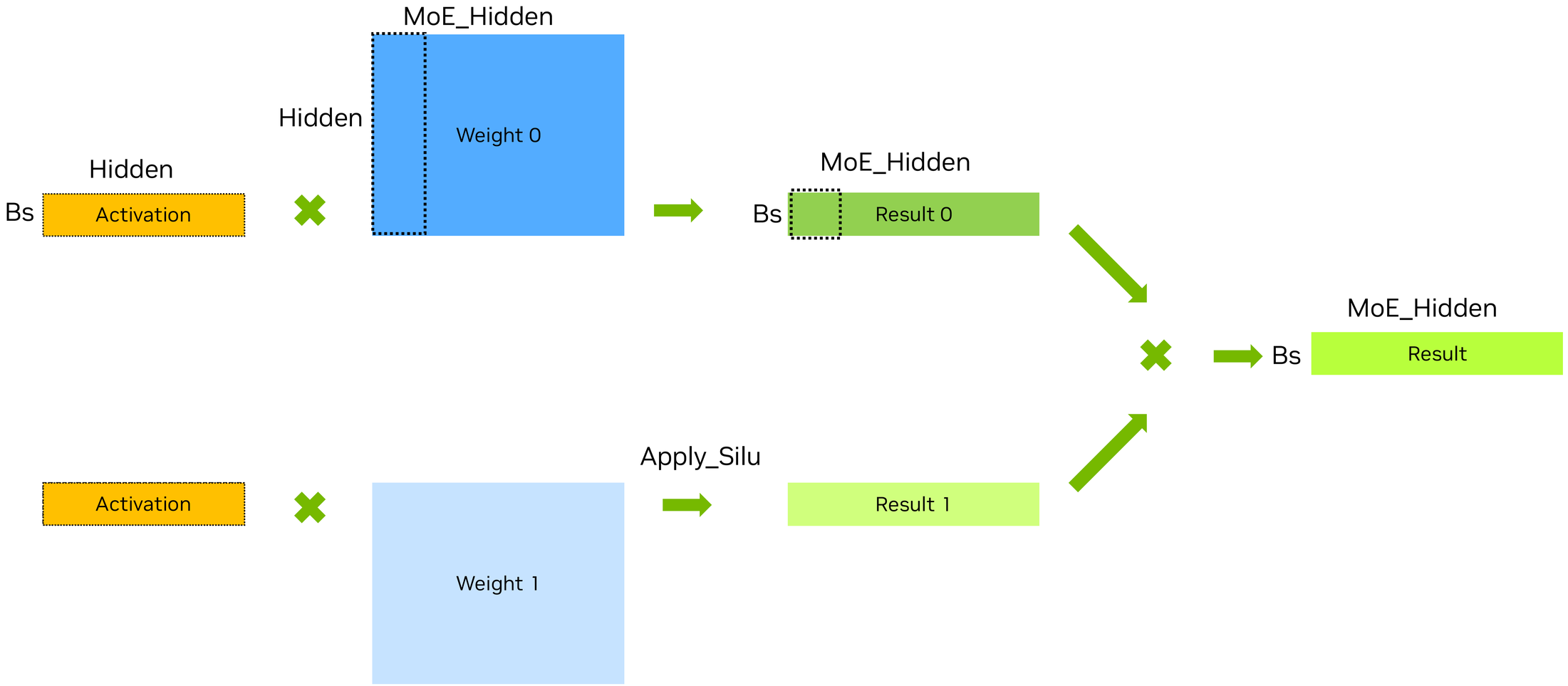
MoE FC1: Data layouts

Combining Gather4 and Runtime Dynamic box size



FC1 (Gated SiLU) in MoE

Per Expert View



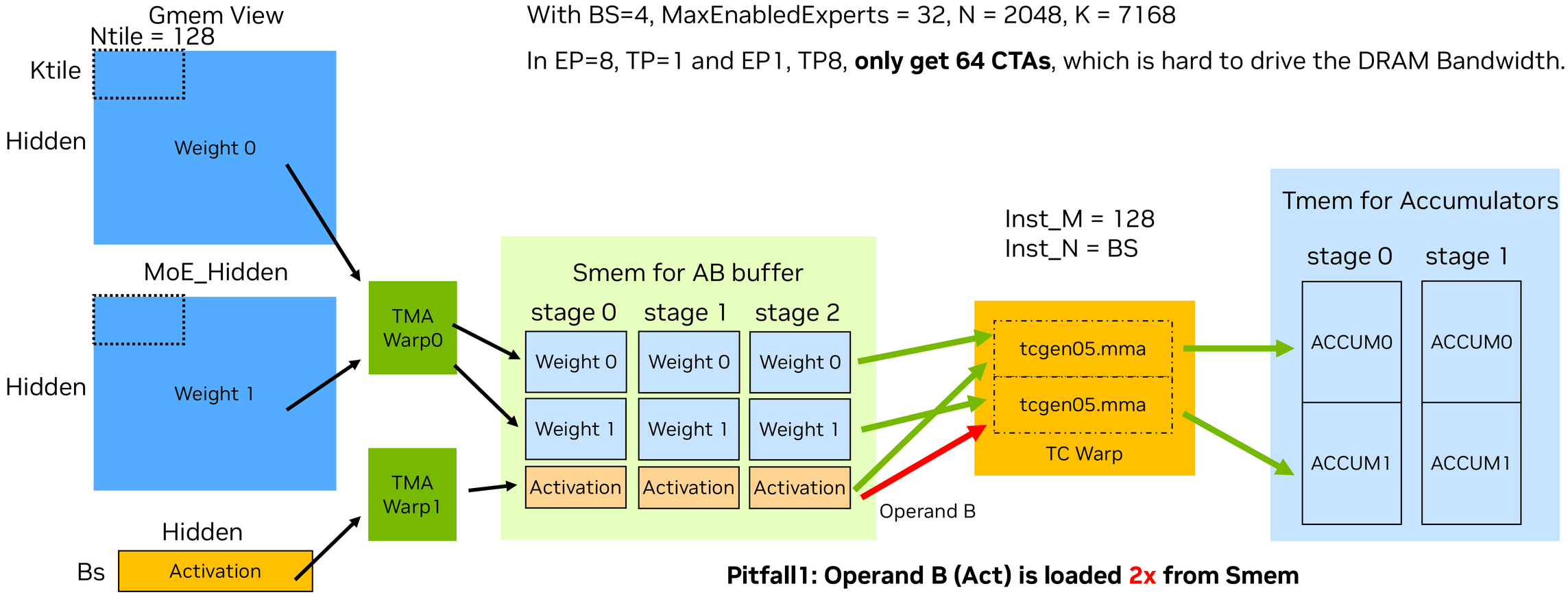
Fused FC1 (Gated SiLU)

Naïve approach.

Pitfall 2: Limited CTAs.

With BS=4, MaxEnabledExperts = 32, N = 2048, K = 7168

In EP=8, TP=1 and EP1, TP8, **only get 64 CTAs**, which is hard to drive the DRAM Bandwidth.



Pitfall1: Operand B (Act) is loaded 2x from Smem

Fused FC1 (Gated SiLU)

Concat 2 weight matrixes in SMEM

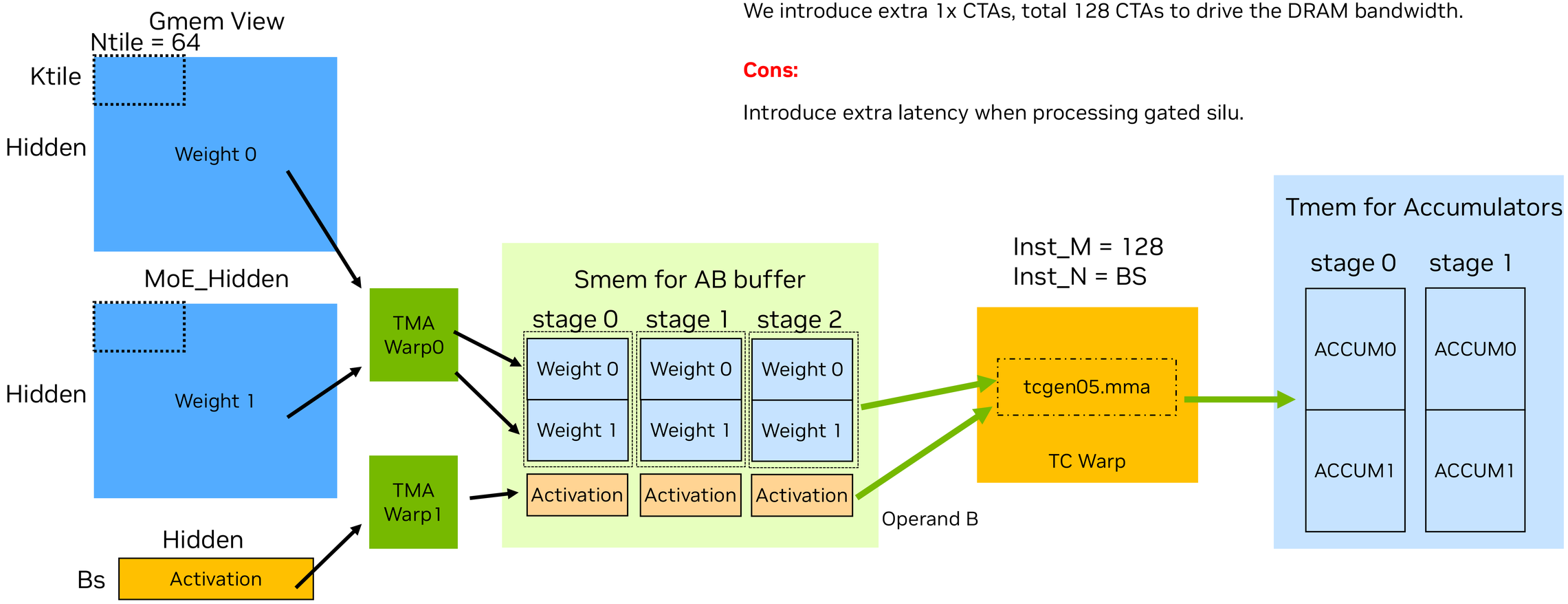
Pros:

1 tcgen05.mma instruction and no duplicated smem loading

We introduce extra 1x CTAs, total 128 CTAs to drive the DRAM bandwidth.

Cons:

Introduce extra latency when processing gated silu.

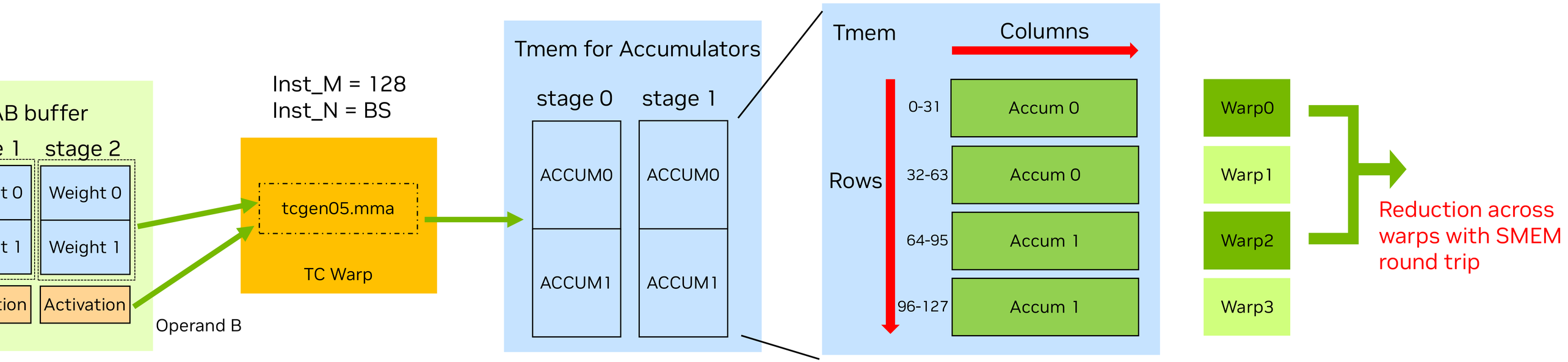


Fused FC1 (Gated SiLU)

Concat 2 weight matrixes in SMEM

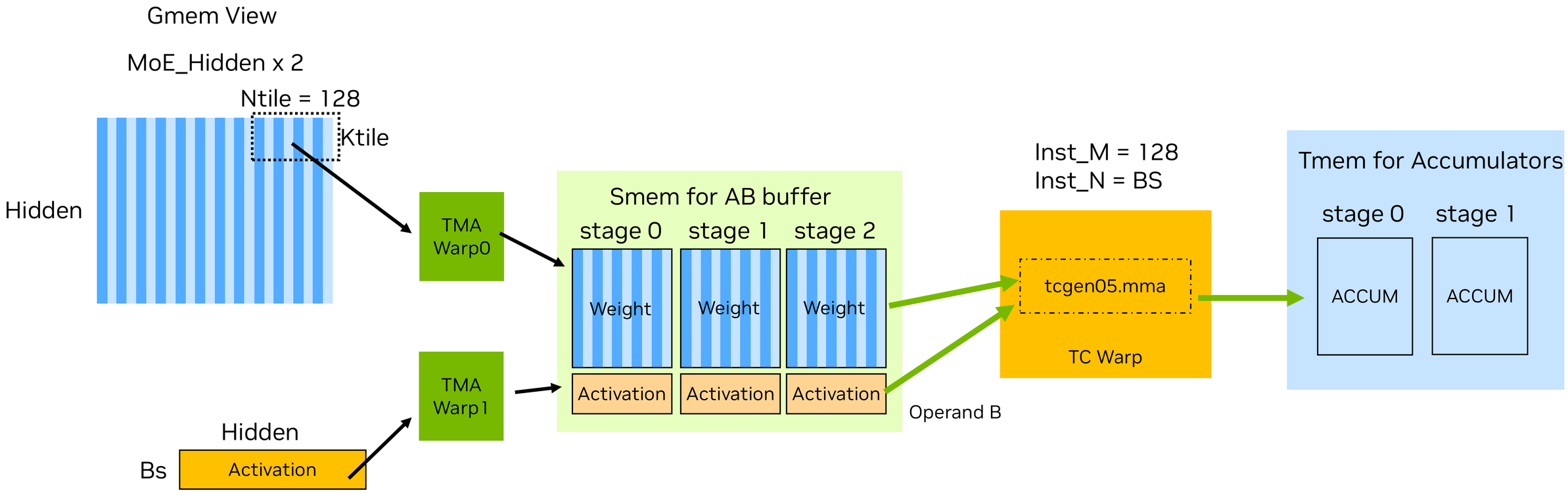
Cons:

Introduce extra latency when processing gated silu.



Interleaved Fused FC1 (Gated SiLU)

Interleave granularity = 1



Interleaved Fused FC1 (Gated SiLU)

Interleave granularity = 1

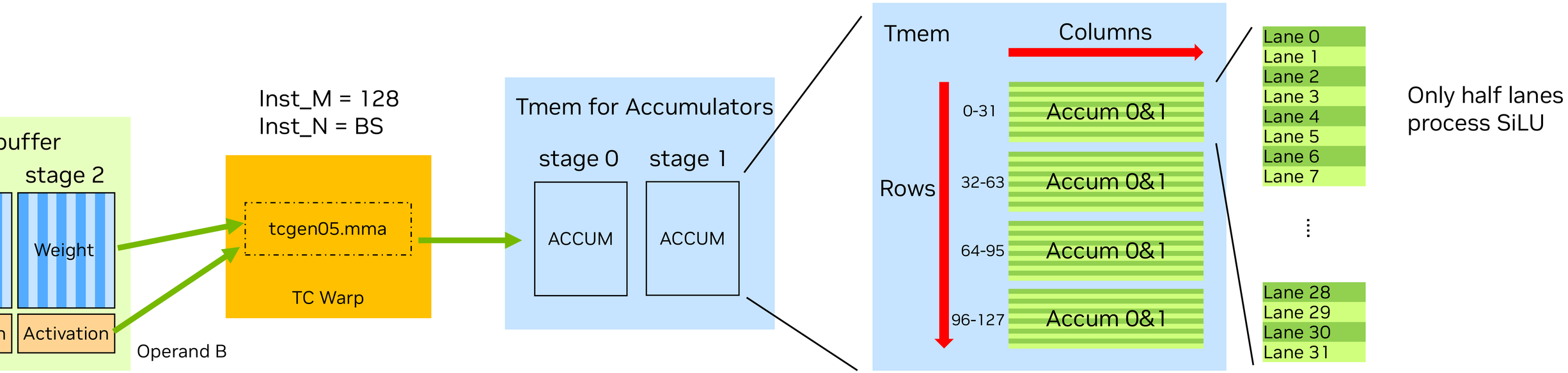
Pros:

Reductions are moved into intra-warp.

Cons:

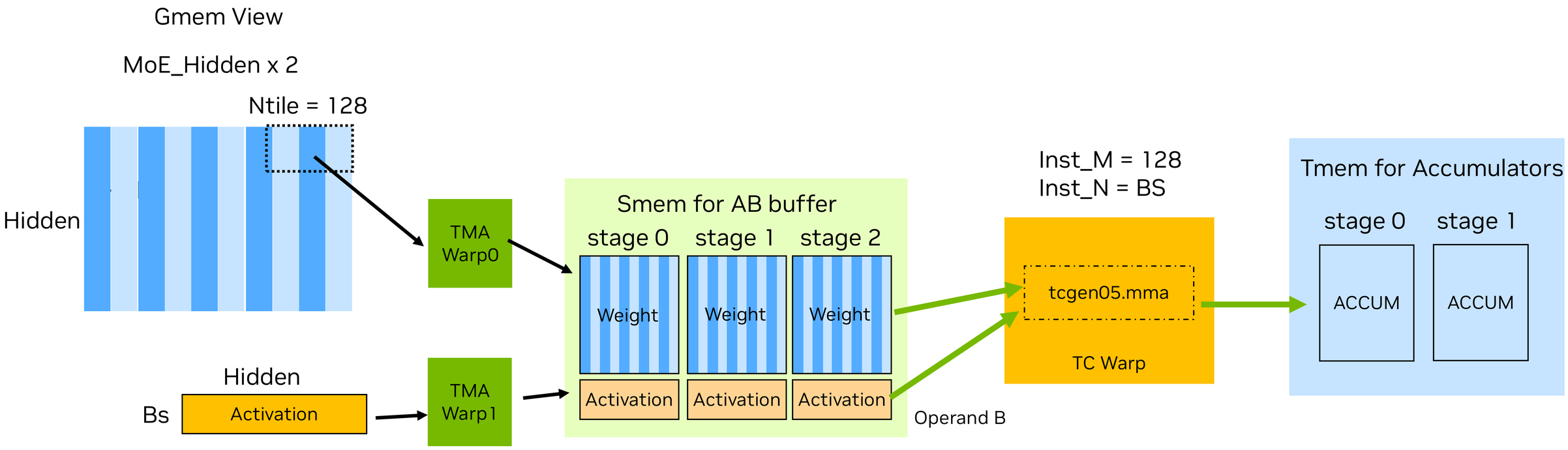
Only half threads perform MUFU instructions, which wastes instruction throughput.

Reduction in 2 adjacent threads is still a little bit expensive.



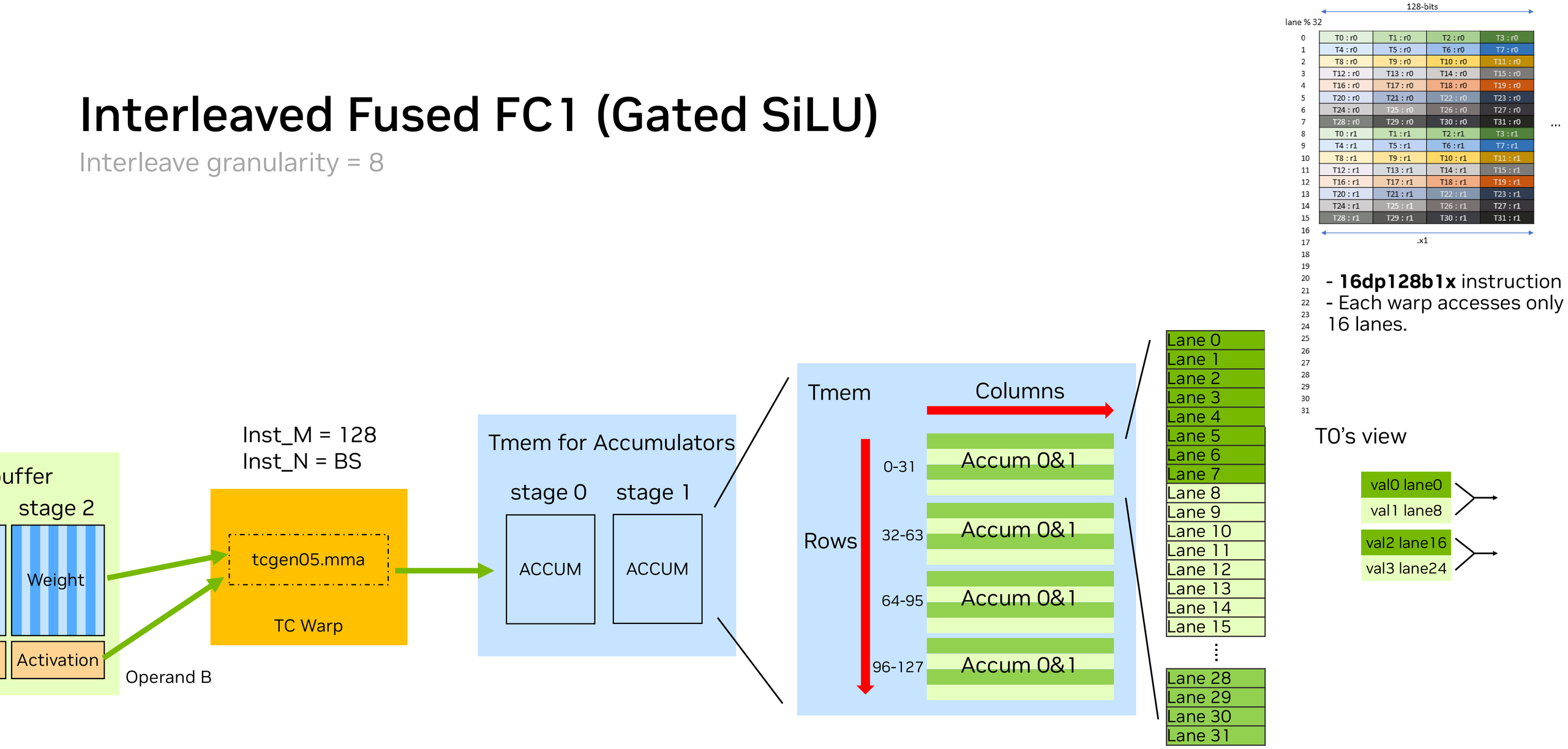
Interleaved Fused FC1 (Gated SiLU)

Interleave granularity = 8



Interleaved Fused FC1 (Gated SiLU)

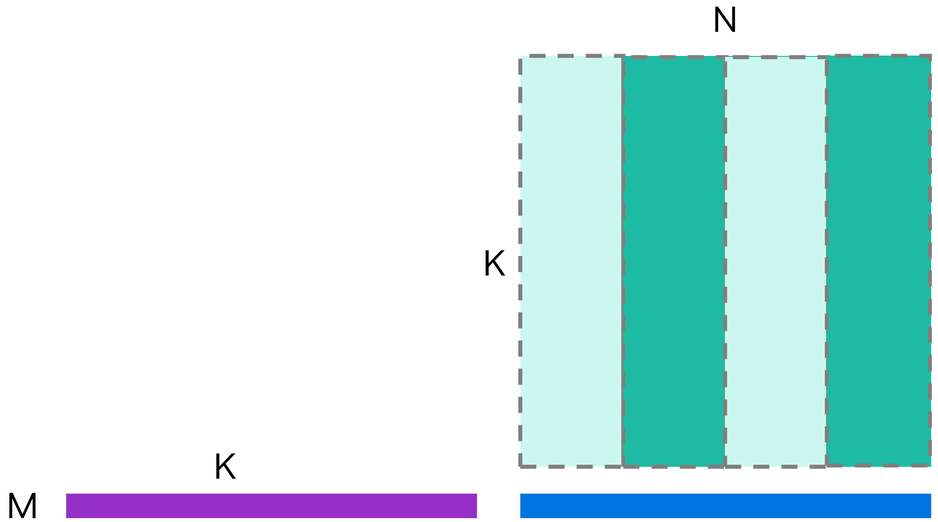
Interleave granularity = 8

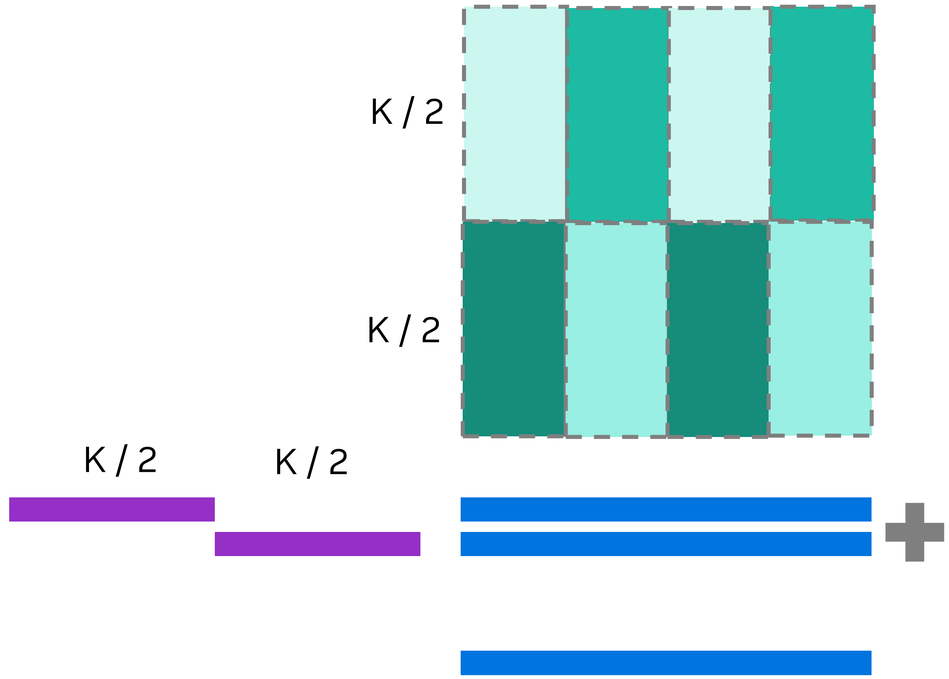


Split-K using block clusters

Split-K

Using Thread-block Clusters





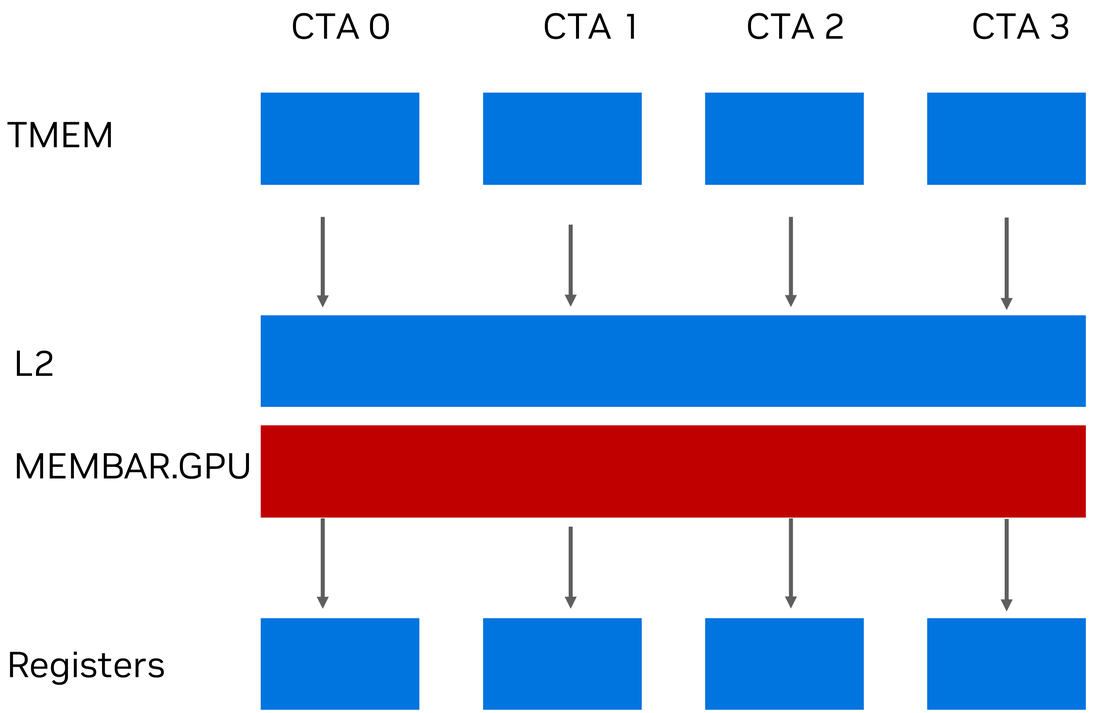
Problem: M and N is small => Grid size is small

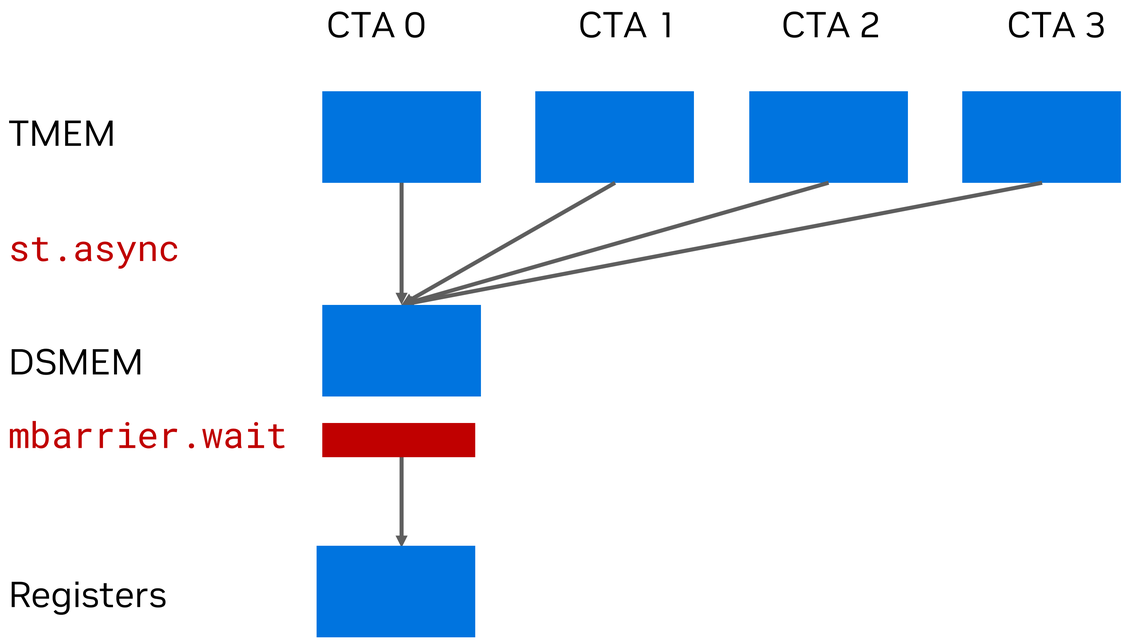
Solution: split up K as well

New problem: how to efficiently accumulate K tiles?

Split-K

Using Thread-block Clusters





L2-based reduction

- Higher latency

Distributed Shared Memory-based reduction

- Lower latency

More suited to **throughput** regime

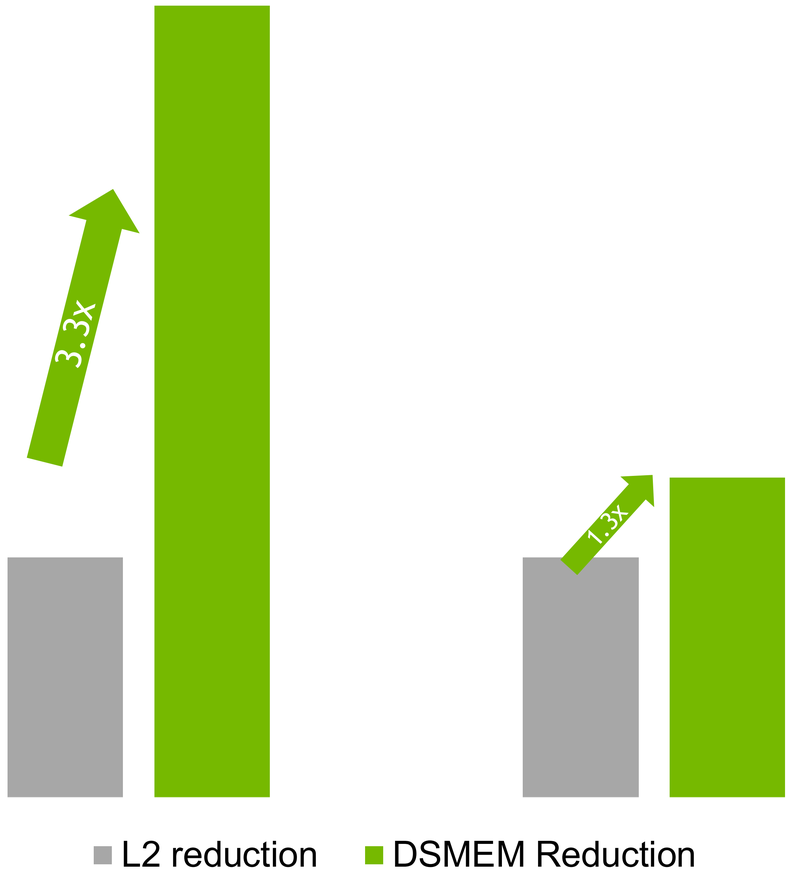
More suited to **latency** regime

For more info: S72683 - CUDA Techniques to Maximize Memory Bandwidth and Hide Latency

Split-K

Results

- Llama 70B QKV GEMM
  - SplitK=8
  - 16x1280x8192



Summary

Summary

Blackwell architecture is well-suited for low-latency LLM inference

System-level features

- CUDA Graph
- Programmatic Dependent Launch
- Fast NVLink for Expert and Tensor Parallelism

Tensor core features

- Low-precision number formats
- Scalable tensor core hardware design

TMA features

- Sparse token loading using gather4
- Dynamic box size